

Introduction to Microservices Architecture

This chapter is the last chapter that talks about application design before we move on to a few user interface chapters. The chapter covers some essential microservices architecture concepts. It is designed to get you started with those principles and give you a good idea of the microservices architecture.

This chapter aims to give you an overview of the concepts surrounding microservices, which should help you make informed decisions about whether you should go for a microservices architecture or not.

Since microservices architecture is larger in scale than the previous application-scale pattern we visited and usually involves advanced components, there is no C# code in the chapter. Instead, I explain the concepts and list open source or commercial offerings that you can leverage to apply these patterns to your applications. Moreover, you should not aim to implement many of the pieces discussed in the chapter because it can be a lot of work to get them right, and they don't add business value, so you are better off just using an existing implementation instead and extending it if needed. There is more context about this throughout the chapter.

That said, monolithic architecture patterns, such as Vertical Slice and Clean Architecture, are still good to know, as you can apply those to individual microservices. Don't worry—all of the knowledge you have acquired since the beginning of this book is not forfeit and is still worthwhile.

The following topics are covered in this chapter:

- What are microservices?
- An introduction to event-driven architecture
- Getting started with message queues
- Implementing the Publish-Subscribe pattern
- Introducing Gateway patterns
- Revisiting the CQRS pattern
- The Microservices Adapter pattern

Let's get started!

What are microservices?

Besides being a buzzword, microservices represent an application that is divided into multiple smaller applications. Each application, or microservice, interacts with the others to create a scalable system. Usually, microservices are deployed to the cloud as containerized or serverless applications.

Before getting into too many details, here are a few principles to keep in mind when building microservices:

- Each microservice should be a cohesive unit of business.
- Each microservice should own its data.
- Each microservice should be independent of the others.

Furthermore, everything we have studied so far—that is, the other principles of designing software—applies to microservices but on another scale. For example, you don't want tight coupling between microservices (solved by microservices independence), but the coupling is inevitable (as with any code). There are numerous ways to solve this problem, such as the Publish-Subscribe pattern.

There are no hard rules about how to design microservices, how to divide them, how big they should be, and what to put where. That being said, I'll lay down a few foundations to help you get started and orient your journey into microservices.

Cohesive unit of business

A microservice should have a single business responsibility. Always design the system with the domain in mind, which should help you divide the application into multiple pieces. If you know **Domain-Driven Design (DDD)**, a microservice will most likely represent a **Bounded Context**, which in turn is what I call a *cohesive unit of business*. Basically, a cohesive unit of business (or bounded context) is a self-contained part of the domain that has limited interactions with other parts of the domain.

Even if a **microservice** has *micro* in its name, it is more important to group logical operations under it than to aim at a micro-size. Don't get me wrong here; if your unit is tiny, that's even better. However, suppose you split a unit of business into multiple smaller parts instead of keeping it together (breaking cohesion).

In that case, you are likely to introduce useless chattiness within your system (coupling between microservices). This could lead to performance

degradation and to a system that is harder to debug, test, maintain, monitor, and deploy.

Moreover, it is easier to split a big microservice into smaller pieces than assemble multiple microservices back together.

Try to apply the SRP to your microservices: a microservice should have only one reason to change unless you have a good reason to do otherwise.

Ownership of data

Each microservice is the source of truth of its cohesive unit of business. A microservice should share its data through an API (a web API/HTTP, for example) or another mechanism (integration events, for example). It should own that data and not share it with other microservices directly at the database level.

For instance, two different microservices should never access the same relational database table. If a second microservice needs some of the same data, it can create its own cache, duplicate the data, or query the owner of that data but not access the database directly; **never**.

This data-ownership concept is probably the most critical part of the microservices architecture and leads to microservices independence. Failing at this will most likely lead to a tremendous number of problems. For example, if multiple microservices can read or write data in the same database table, each time something changes in that table, all of them must be updated to reflect the changes. If different teams manage the microservices, that means cross-team coordination. If that happens, each microservice is not independent anymore, which opens the floor to our next topic.

Microservice independence

At this point, we have microservices that are cohesive units of business and own their data. That defines **independence**.

This independence offers the systems the ability to scale while having minimal to no impact on the other microservices. Each microservice can also scale independently, without the need for the whole system to be scaled. Additionally, when the business requirements grow, each part of that domain can evolve independently.

Furthermore, you could update one microservice without impacting the others or even have a microservice go offline without the whole system stopping.

Of course, microservices have to interact with one another, but the way they do should define how well your system runs. A little like Vertical Slice architecture, you are not limited to using one set of architectural patterns; you can independently make specific decisions for each microservice. For example, you could choose a different way for how two microservices communicate with each other versus two others. You could even use different programming languages for each microservice.

Tip

I recommend sticking to one or a few programming languages for smaller businesses and organizations as you most likely have fewer developers, and each has more to do. Based on my experience, you want to ensure business continuity when people leave and make sure you can replace them and not sink the ship due to some obscure technologies used here and there (or too many technologies).

Now that we've defined the basics, let's jump into the different ways microservices can communicate using event-driven architecture. We first explore ways to mediate communication between microservices using message queues and the Publish-Subscribe pattern. We then learn ways to shield and hide the complexity of the microservices cluster using Gateway patterns. After that, we dig into more detail about the CQRS pattern and provide a conceptual serverless example.

An introduction to event-driven architecture

Event-driven architecture (EDA) is a paradigm that revolves around consuming streams of events, or data in motion, instead of consuming static states.

What I define by a static state is the data stored in a relational database table or other types of data stores, like a NoSQL documents store. That data is dormant in a central location and waiting for actors to consume and mutate it. It is stale between every mutation and the data (a record, for example) represents a finite state.

On the other hand, data in motion is the opposite: you consume the ordered events and determine the change in state that each event brings.

What is an event? People often interchange the words event, message, and command. Let's try to clarify this:

- A message is a piece of data that represents something.
- A message can be an object, a JSON string, bytes, or anything else your system can interpret.
- An event is a message that represents something that happened in the past.
- A command is a message sent to tell one or more recipients to do something.
- A command is sent (past tense), so we can also consider it an event.

A message usually has a payload (or body), headers (metadata), and a way to identify it (this can be through the body or headers).

We can use events to divide a complex system into smaller pieces or have multiple systems talk to each other without creating tight couplings. Those systems could be subsystems or external applications, such as microservices.

Like **Data Transfer Objects (DTO)** of web APIs, events become the data contracts that tie the multiple systems together (coupling). It is essential to think about that carefully when designing events. Of course, we cannot foresee the future, so we can only do so much to get it perfect the first time. There are ways to version events, but this is out of the scope of this chapter.

EDA is a fantastic way of breaking tight coupling between microservices but requires rewiring your brain to learn this newer paradigm. Tooling is less mature, and expertise is scarcer than more linear ways of thinking (like using point-to-point communication and relational databases), but this is slowly changing and well worth learning (in my opinion).

Before moving further, we can categorize events into the following overlapping buckets:

- Domain events
- Integration events
- Application events
- Enterprise events

As we explore next, all types of events play a similar role with different intents and scopes.

Domain events

A domain event is a term based on DDD representing an event in the domain. This event could then trigger other pieces of logic to be executed subsequently. It allows a complex process to be divided into multiple smaller processes. Domain events work well with domain-centric designs, like Clean Architecture, as we can use them to split complex domain objects into multiple smaller pieces. Domain events are usually application events. We can use MediatR to publish domain events inside an application.

To summarize, **domain events integrate pieces of domain logic together while keeping the domain logic segregated**, leading to loosely coupled components that hold one domain responsibility each (single responsibility principle).

Integration events

Integration events are like domain events but are used to propagate messages to external systems, to integrate multiple systems together while keeping them independent. For example, a microservice could send the `new user registered` event message that other microservices react to, like saving the `user id` to enable additional capabilities or sending a greeting email to that new user.

We use a message broker or message queue to publish such events. We cover those next, after covering application and enterprise events.

To summarize, **integration events integrate multiple systems together while keeping them independent**.

Application events

An application event is an event that is internal to an application; it is just a matter of scope. If the event is internal to a single process, that event is also a domain event (most likely). If the event crosses microservices boundaries that your team owns (the same application), it is also an integration event. The event itself won't be different; it is the reason why it exists and its scope that describes it as an application event or not.

To summarize, **application events are internal to an application**.

Enterprise events

An enterprise event describes an event that crosses internal enterprise boundaries. These are tightly coupled with your organizational structure. For example, a microservice sends an event that other teams, part of other divisions or departments, consume.

The governance model around those events should be different from application events that only your team consumes.

Someone must think about who can consume that data, under what circumstances, the impact of changing the event schema (data contract), schema ownership, naming conventions, data-structure conventions, and more, or risk building an unstable data highway.

Note

I like to see EDA as a central **data highway** in the middle of applications, systems, integrations, and organizational boundaries, where the events (data) flow between systems in a loosely coupled manner.

It's like a highway where cars flow between cities (without traffic jams). The cities are not controlling what car goes where but are open to visitors.

To summarize, **enterprise events are integration events that cross organizational boundaries.**

Conclusion

We defined events, messages, and commands in this quick overview of event-driven architecture. An event is a snapshot of the past, a message is data, and a command is an event that suggests other systems to take action. Since all messages are from the past, calling them events is accurate. We then organized events into a few overlapping buckets to help identify the intents. We can send events for different objectives, but whether it is about designing independent components or reaching out to different parts of the business, an event remains a payload that respects a certain format (schema). That schema is the data contract (coupling) between the consumers of those events. That data contract is probably the most important piece of it all; break the contract, break the system.

Now, let's see how event-driven architecture can help us follow the **SOLID** principles at cloud-scale:

- **S:** Systems are independent of each other by raising and responding to events. The events themselves are the glue that ties those systems together. Each piece has a single responsibility.
- **O:** We can modify the system's behaviors by adding new consumers to a particular event without impacting the other applications. We can also raise new events to start building a new process without affecting existing applications.
- **L:** N/A
- **I:** Instead of building a single process, EDA allows us to create multiple smaller systems that integrate through data contracts (events) where those contracts become the messaging interfaces of the system.
- **D:** EDA enables systems to break tight coupling by depending on the events (interfaces/abstractions) instead of communicating directly with one another, inverting the dependency flow.

EDA does not only come with advantages; it also has a few drawbacks that we explore in subsequent sections of the chapter.

Next, we explore message queues followed by the Publish-Subscribe pattern, two ways of interacting with events.

Getting started with message queues

A **message queue** is nothing more than a queue that we leverage to send ordered messages. A queue works on a **First In, First Out (FIFO)** basis. If our application runs in a single process, we could use one or more `Queue<T>` instances to send messages between our components or a `ConcurrentQueue<T>` instance to send messages between threads. Moreover, queues can be managed by an independent program to send messages in a distributed fashion (between applications or microservices).

A distributed message queue can add more or fewer features to the mix, which is especially true for cloud programs that have to handle failures at more levels than a single server does. One of those features is the **dead letter queue**, which stores messages that failed some criteria in another queue. For example, if the target queue is full, a message could be sent to the **dead letter queue** instead. One could requeue such messages by putting the message back at the end of the queue (beware, this changes the initial order in which messages were sent).

Many messaging queue protocols exist; some are proprietary, while others are open source. Some messaging queues are cloud-based and used as a service, such as Azure Service Bus and Amazon Simple Queue Service. Others are open source and can be deployed to the cloud or on-premises, such as Apache ActiveMQ.

If you need to process messages in order and want each message to be delivered to a single recipient at a time, a **message queue** seems like the right choice. Otherwise, the **Publish-Subscribe** pattern could be a better fit for you.

Here is a basic example that illustrates what we just discussed:

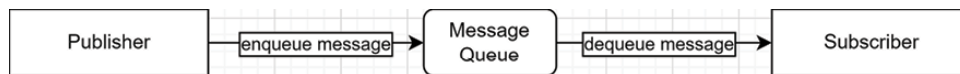


Figure 16.1: A publisher that enqueues a message with a subscriber that dequeues it

For a more concrete example, in a distributed user registration process, when a user registers, we could want to do the following:

- Send a confirmation email.
- Process their picture and save one or more thumbnails.
- Send an onboarding message to their in-app mailbox.

To sequentially achieve this, one operation after the other, we could do the following:

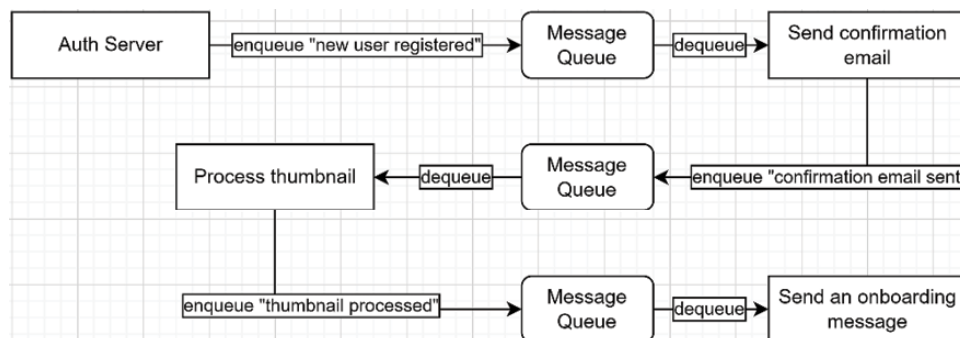


Figure 16.2: A process flow that sequentially executes three operations that happen after a user creates an account

In this case, if the process crashes during the *Process Thumbnail* operation, the user would not receive the *Onboarding Message*. Another drawback would be that to insert a new operation between the *Process Thumb-*

nail and *Send an onboarding message* steps, we'd have to modify the *Send an onboarding message* operation (tight coupling).

If the order does not matter, we could queue all the messages from the *Auth Server* instead, right after the user's creation, like this:

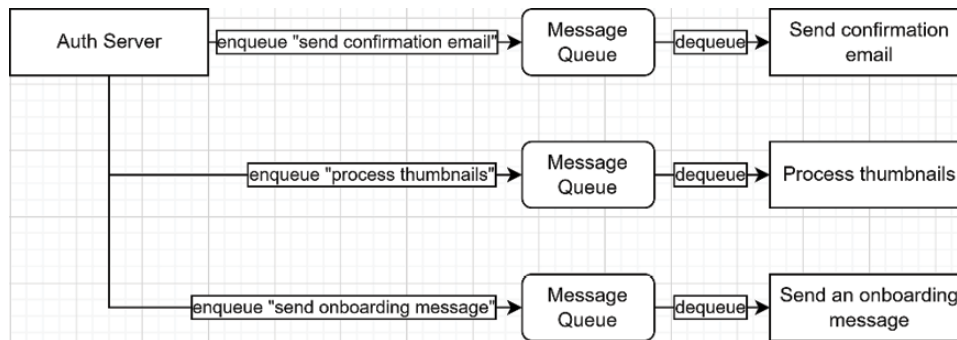


Figure 16.3: The Auth Server is queuing the operations sequentially while different processes execute them in parallel

This process is better, but the *Auth Server* is now controlling what should be happening once a new user has been created. The *Auth Server* was queuing an event in the previous workflow that told the system that a new user was registered. However, now, it has to be aware of the post-processing workflow to queue each operation sequentially. Doing this is not wrong in itself and is easier to follow when you dig into the code, but it creates tighter coupling between the services where the *Auth Server* is aware of the external processes.

According to the SRP, I don't see how an authentication/authorization server should be responsible for anything other than authentication, authorization, and managing that data.

If we continue from there and want to add a new operation between two existing steps, we would only have to modify the *Auth Server*, which is less error-prone than the preceding workflow.

If we want the best of both worlds, we could use the **Publish-Subscribe** pattern instead, which we cover next. We revisit this example there.

Conclusion

If you need messages to be delivered sequentially, a queue might well be the right tool to use. The example that we explored was “doomed to failure” from the beginning, but it allowed us to explore the thinking process behind designing the system. Sometimes, the first idea is not the best and

can be improved by exploring new ways of doing things or learning new skills. Being open-minded to the ideas of others can also lead to better solutions.

Message queues are amazing at buffering messages for high-demand scenarios where an application may not be able to handle spikes of traffic. In that case, the messages are enqueued so the application can catch up at its own speed, reading them sequentially.

Implementing distributed message queues requires a lot of knowledge and effort and is not worth it for almost all scenarios. The big cloud providers like AWS and Azure offer fully managed message queue systems as a service. You can also look at **ActiveMQ**, **RabbitMQ**, or any **Advanced Message Queuing Protocol (AMQP)** broker.

One essential aspect of choosing the right queue system is whether you are ready and have the skills to manage your own distributed message queue. If you want to speed up development and have enough money on hand, you should use a fully managed offering for at least your production environment, especially if you are expecting a large volume of messages. On the other hand, using a local or on-premise instance for development or smaller scale usage may save you a considerable sum of money. Choosing an open source system with fully managed cloud offerings is a good way to achieve both: low local development cost with an always available high-performance cloud production offering that the service provider maintains for you.

Another aspect is to base your choice on needs. Have clear requirements and ensure the system you choose does what you need. Some offerings can also cover multiple use cases like queues and pub-sub (which we are exploring next), leading to learning or requiring fewer skills that enable more possibilities.

Before moving to the next pattern, let's see how message queues can help us follow the **SOLID** principles at the app scale:

- **S**: Helps centralize and divide responsibilities between applications or components without them directly knowing each other, breaking tight coupling.
- **O**: Allows us to change the message producer's or subscriber's behaviors without the other knowing about it.
- **L**: N/A
- **I**: Each message and handler can be as small as needed, while each microservice indirectly interacts with the others to solve the bigger

problem.

- **D:** By not knowing the other dependencies (breaking tight coupling between microservices), each microservice depends only on the messages (abstractions) instead of concretions (the other microservices API).

One drawback is the delay between enqueueing a message and processing a message. We talk about delay and latency in more detail in subsequent sections.

Implementing the Publish-Subscribe pattern

The **Publish-Subscribe** pattern (Pub-Sub) is very similar to what we did using **MediatR** and what we explored in the *Getting started with message queues* section. However, instead of sending one message to one handler (or enqueueing a message), we publish (send) a message (event) to zero or more subscribers (handlers). Moreover, the publisher is unaware of the subscribers; it only sends messages out, hoping for the best (also known as **fire and forget**).

Note

Using a message queue does not mean you are limited to only one recipient.

We can use **Publish-Subscribe** in-process or in a distributed system through a **message broker**. The message broker is responsible for delivering the messages to the subscribers. That is the way to go for microservices and other distributed systems since they are not running in a single process.

This pattern has many advantages over other ways of communication. For example, we could recreate the state of a database by replaying the events that happened in the system, leading to the **event sourcing** pattern. More on that later.

The design depends on the technology that's used to deliver the messages and the configuration of that system. For example, you could use **MQTT** to deliver messages to **Internet of Things (IoT)** devices and configure them to retain the last message sent on each topic. That way, when a device connects to a topic, it receives the latest message. You could also configure a **Kafka** broker that keeps a long history of messages and asks for

all of them when a new system connects to it. All of that depends on your needs and requirements.

MQTT and Apache Kafka

If you were wondering what MQTT is, here is a quote from their website <https://adpg.link/mqtt>:

“MQTT is an OASIS standard messaging protocol for the Internet of Things (IoT). It is designed as an extremely lightweight publish/subscribe messaging transport [...]”

Here is a quote from Apache Kafka’s website <https://adpg.link/kafka>:

“Apache Kafka is an open-source distributed event streaming platform [...]”

We cannot cover every single scenario of every single system that follows every single protocol. Therefore, I’ll highlight some shared concepts behind the Pub-Sub design pattern so that you know how to get started. Then, you can dig into the specific technology that you want (or need) to use.

A topic is a way to organize events, a channel, a place to read or write specific events so consumers know where to find them. As you can probably imagine, sending all events to the same place is like creating a relational database with a single table: it would be suboptimal, hard to manage, use, and evolve.

To receive messages, subscribers must subscribe to topics (or the equivalent of a topic):

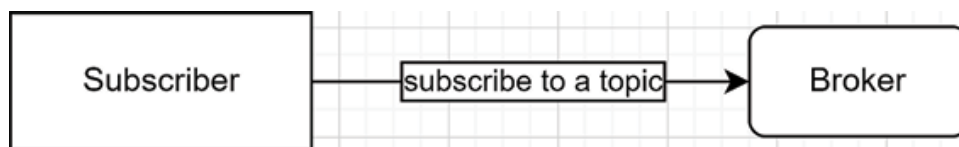


Figure 16.4: A subscriber subscribes to a pub-sub topic

The second part of the Pub-Sub pattern is to publish messages, like this:

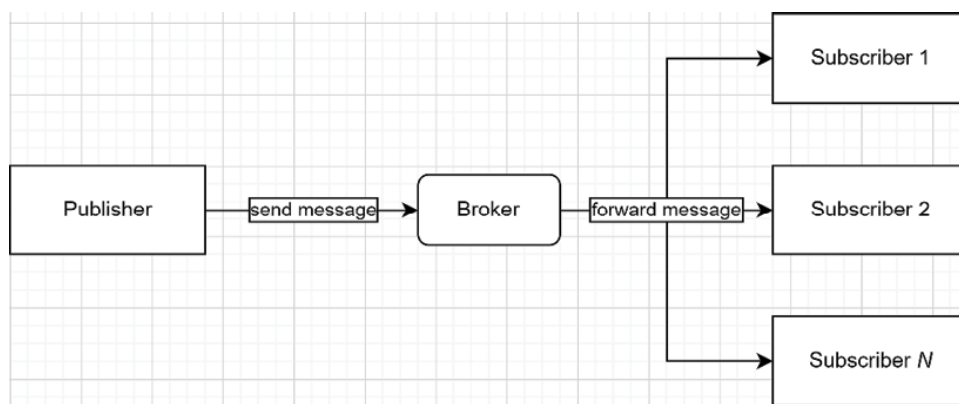


Figure 16.5: A publisher is sending a message to the message broker. The broker then forwards that message to N subscribers, where N can be zero or more

Many abstracted details here depend on the broker and the protocol. However, the following are the two primary concepts behind the Publish-Subscribe pattern:

- Publishers publish messages to topics.
- Subscribers subscribe to topics to receive messages when they are published.

Note

For example, one crucial implementation detail that is not illustrated here is security. Security is mandatory in most systems, and not every subsystem or device should have access to all topics.

Publishers and subscribers could be any part of any system. For example, many Microsoft Azure services are publishers (for example, Blob storage). You can then have other Azure services (for example, Azure Functions) subscribe to those events and react to them.

You can also use the **Publish-Subscribe** pattern inside your applications—there’s no need to use cloud resources for that; this can even be done inside the same process.

The most significant advantage of the Publish-Subscribe pattern is breaking tight coupling between systems. One system publishes events while others consume them without the systems knowing each other.

That loose coupling leads to scalability, where each system can scale independently and where messages can be processed in parallel using the re-

sources it requires. It is easier to add new processes to a workflow as well since the systems are unaware of the others. To add a new process that reacts to an event, you only have to create a new microservice, deploy it, then start to listen to one or more events and process them.

On the downside, the message broker can become the application's single point of failure and must be configured appropriately. It is also essential to consider the best message delivery policies for each message type. An example of a policy could be to ensure the delivery of crucial messages while delaying less time-sensitive messages and dropping unimportant messages during load surges.

If we revisit our previous example using Publish-Subscribe, we end up with the following simplified workflow:

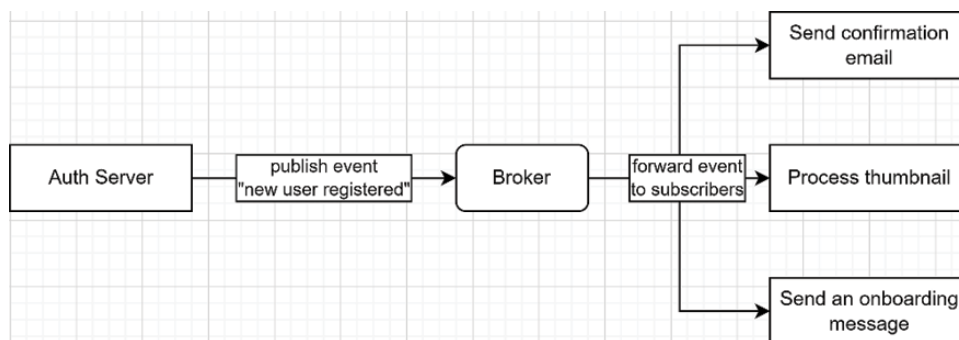


Figure 16.6: The Auth Server is publishing an event representing the creation of a new user. The broker then forwards that message to the three subscribers that then execute their tasks in parallel

Based on this workflow, we decoupled the Auth Server from the post-registration process. The Auth Server is not aware of the workflow, and the individual services are unaware of each other. Moreover, if we want to add a new task, we only have to create or update a microservice that subscribes to the right topic (in this case, the “new user registered” topic).

The current system does not support synchronization and does not handle process failures or retries, but it is a good start since we combine the pros of the message queue examples and leave the cons behind.

Now that we have explored the Publish-Subscribe pattern, we look at message brokers, then dig deeper into EDA and leverage the Publish-Subscribe pattern to create a persistent database of events that can be replayed: the Event Sourcing pattern.

Message brokers

A message broker is a program that allows us to send (**publish**) and receive (**subscribe**) messages. It plays the mediator role at scale, allowing multiple applications to talk to each other without knowing each other (**loose coupling**). The message broker is usually the central piece of any event-based distributed system that's implementing the publish-subscribe pattern.

An application (**publisher**) publishes messages to topics, while other applications (**subscribers**) receive messages from those topics. The notion of **topics** may differ from one protocol or system to another, but all systems I know have a topic-like concept to route messages to the right place. For example, you can publish to the `Devices` topic using Kafka, but to `devices/abc-123/do-something` using MQTT.

How you name your topics depends significantly on the system you are using and the scale of your installation. For example, MQTT is a lightweight event broker that recommends using a path-like naming convention. On the other hand, Apache Kafka is a full-featured event broker and event streaming platform that is not opinionated about topic names, leaving you in charge of that. Depending on the scale of your implementation, you can use the entity name as the topic name or may need prefixes to identify who in the enterprise can interact with what part of the system. Due to the small scale of the examples of the chapter, we stick with simple topic names, which also makes the examples easier to understand.

The message broker is responsible for forwarding the messages to the registered recipients. The lifetime of those messages can vary by broker or even per individual message or topic.

There are multiple message brokers out there using different protocols. Some brokers are cloud-based, such as Azure Event Grid. Other brokers are lightweight and more suited for IoT, such as Eclipse Mosquitto/MQTT. In contrast to MQTT, others are more robust and allow for high-velocity streaming of data, such as Apache Kafka.

What message broker to use should be based on the requirements of the software that you are building. Moreover, you are not limited to one broker. Nothing stops you from picking a message broker that handles the dialogs between your microservices and uses another to handle the dialogs with external IoT devices. If you are building a system in Azure, want to go serverless, or prefer paying for SaaS components that scale without investing maintenance time, you can leverage Azure services such as Event Grid, Service Bus, and Queue Storage. If you prefer open source software, you can choose Apache Kafka and even run a fully managed cloud in-

stance as a service using Confluent Cloud if you don't want to manage your own cluster.

The event sourcing pattern

Now that we have explored the Publish-Subscribe pattern, learned what an event is, and talked about event brokers, it is time to explore **how to replay the state of an application**. To achieve this, we can follow the **event sourcing pattern**.

The idea behind event sourcing is to **store a chronological list of events** instead of a single entity, where that collection of events becomes the source of truth. That way, every single operation is saved in the right order, helping with concurrency. Moreover, we could replay all of these events to generate an object's current state in a new application, allowing us to deploy new microservices more easily.

Instead of just storing the data, if the system propagates it using an event broker, other systems can cache some of the data as one or more **materialized views**.

Materialized views

A materialized view is a model that's created and stored to serve a specific purpose. The data can come from one or more sources, leading to improved performance when querying that data. For example, the application returns the materialized view instead of querying multiple other systems to acquire the data. You can view the materialized view as a cached entity that a microservice stores in its own database.

One of the drawbacks of event sourcing is data consistency. There is an unavoidable delay between when a service adds an event to the store and when all the other services update their materialized views. This is named **eventual consistency**.

Eventual consistency

Eventual consistency means that the data will be consistent at some point in the future, but not outright. The delay can be from a few milliseconds to a lot longer, but the goal is usually to keep that delay as small as possible.

Another drawback is the complexity of creating such a system compared to a single application that queries a single database. Like the microservices architecture, event sourcing is not just rainbows and unicorns. It comes at a price: **operational complexity**.

Operational complexity

In a microservices architecture, each piece is smaller, but gluing them together has a cost. For example, the infrastructure to support microservices is more complex than a monolith (one app and one database). The same goes for event sourcing; all applications must subscribe to one or more events, cache data (materialized view), publish events, and more. This operational complexity represents the shift of complexity from the application code to the operational infrastructure. In other words, it requires more work to deploy and maintain multiple microservices and databases, as well as to fight the possible instability of network communication between those external systems than it does for a single application containing all of the code. Monoliths are simpler: they work or don't; they rarely partially work.

A crucial aspect of event sourcing is appending new events to the store and never changing existing events (append-only). In a nutshell, microservices communicating using the Pub-Sub pattern publish events, subscribe to topics, and generate materialized views to serve their clients.

Example

Let's explore an example of what could happen if we mix what we just studied together. **Context:** We need to build a program that manages IoT devices. We begin by creating two microservices:

- The `DeviceTwin` microservice, which handles an IoT device's twin's data (that is, a digital representation of the device).
- The `Networking` microservice, which manages the networking-related information of IoT devices (that is, how to reach a device).

As a visual reference, the final system could look as follows (we cover the `DeviceLocation` microservice later):

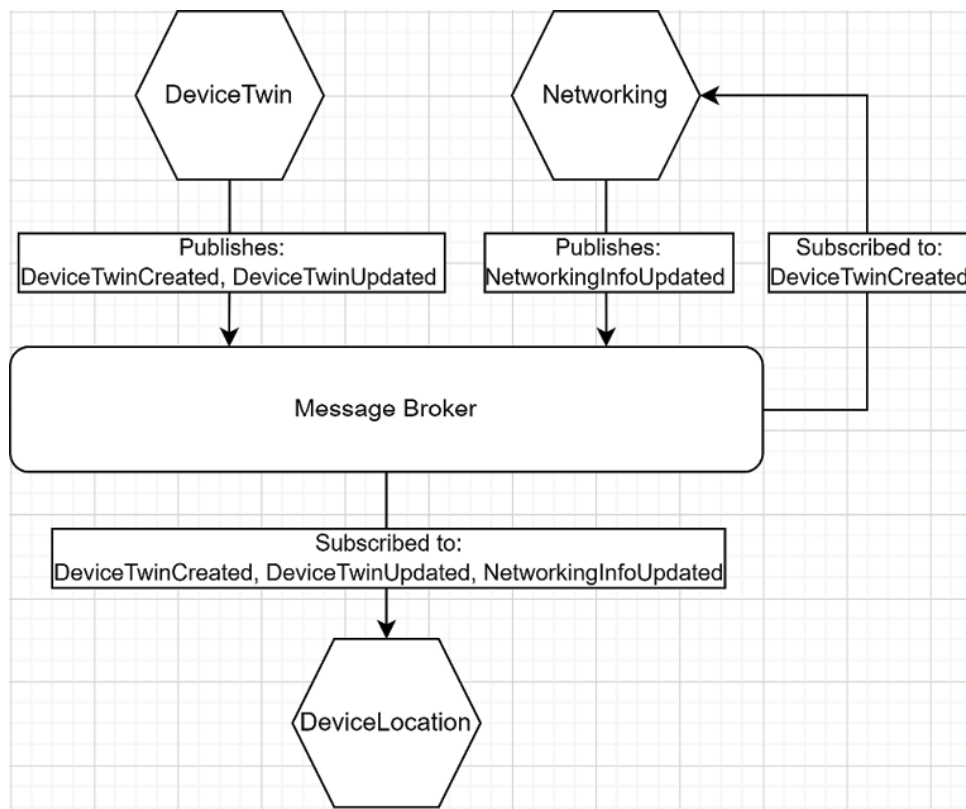


Figure 16.7: Three microservices communicating using the Publish-Subscribe pattern

Here are the user interactions and the published events:

1. A user creates a twin in the system named Device 1. The `DeviceTwin` microservice saves the data and publishes the `DeviceTwinCreated` event with the following payload:

```
{
  "id": "some id",
  "name": "Device 1",
  "other": "properties go here..."
}
```

In parallel, the `Networking` microservice needs to know when a device is created, so it subscribed to the `DeviceTwinCreated` event. When a new device is created, the `Networking` microservice creates default networking information for that device in its database; the default is `unknown`. This way, the `Networking` microservice knows what devices exist or not:

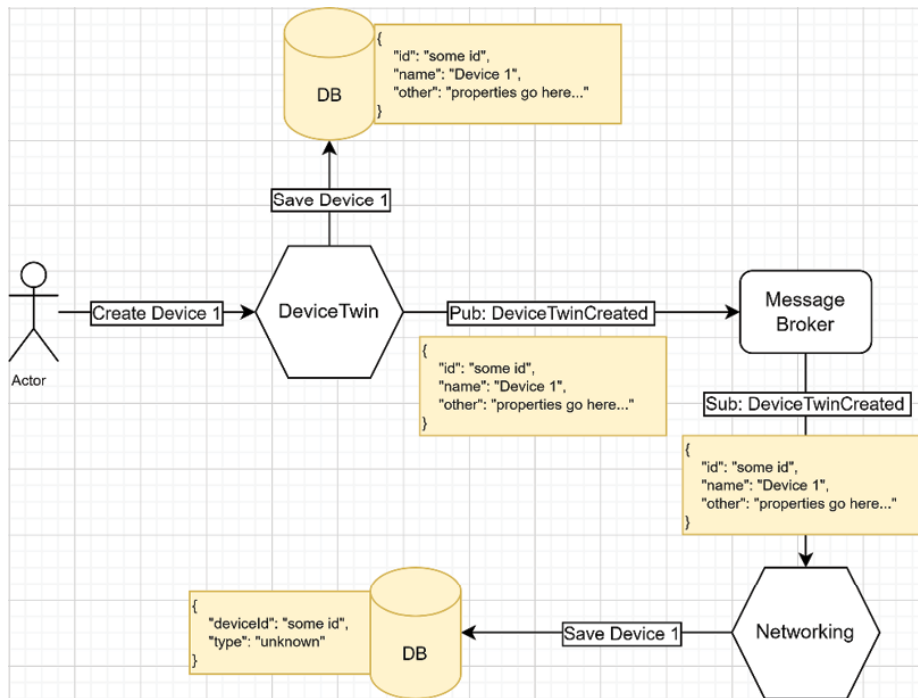


Figure 16.8: A workflow representing the creation of a device twin and its default networking information

2. A user then updates the networking information of that device and sets it to MQTT. The Networking microservice saves the data and publishes the NetworkingInfoUpdated event with the following payload:

```

{
  "deviceId": "some id",
  "type": "MQTT",
  "other": "networking properties..."
}
  
```

This is demonstrated by the following diagram:

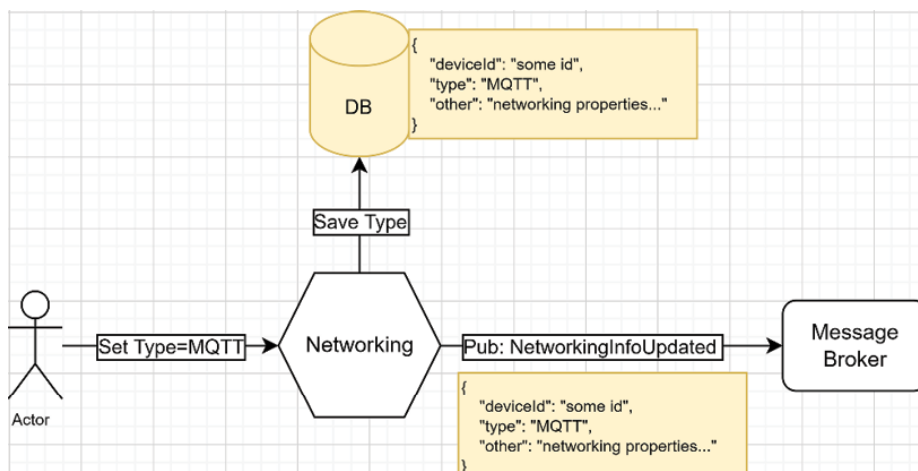


Figure 16.9: A workflow representing updating the networking type of a device

3. A user changes the device's display name to `Kitchen Thermostat`, which is more relevant. The `DeviceTwin` microservice saves the data and publishes the `DeviceTwinUpdated` event with the following payload. The payload uses **JSON patch** to publish only the differences instead of the whole object (see the *Further reading* section for more information):

```
{
  "id": "some id",
  "patches": [
    { "op": "replace", "path": "/name", "value": "Kitchen Thermost"
  ]
}
```

This is demonstrated by the following diagram:

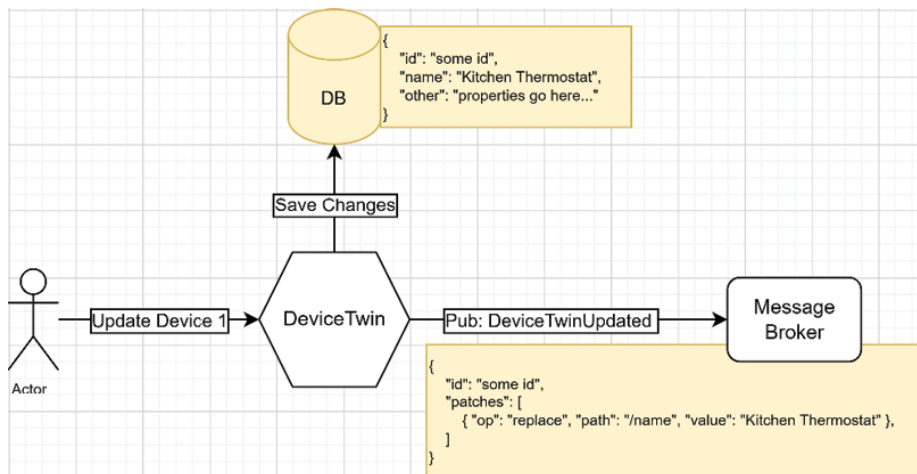


Figure 16.10: A workflow representing a user updating the name of the device to *Kitchen Thermostat*

From there, let's say another team designed and built a new microservice that organizes the devices at physical locations. This new `DeviceLocation` microservice allows users to visualize their devices' location on a map, such as a map of their house.

The `DeviceLocation` microservice subscribes to all three events to manage its materialized view, like this:

- When receiving a `DeviceTwinCreated` event, it saves its unique identifier and display name.

- When receiving a `NetworkingInfoUpdated` event, it saves the communication type.
- When receiving a `DeviceTwinUpdated` event, it updates the device's display name.

When the service is deployed for the first time, it replays all events from the beginning (**event sourcing**); here is what happens:

1. `DeviceLocation` receives the `DeviceTwinCreated` event and creates the following model for that object:

```
{
  "device": {
    "id": "some id",
    "name": "Device 1"
  },
  "networking": {},
  "location": {...}
}
```

This is demonstrated by the following diagram:

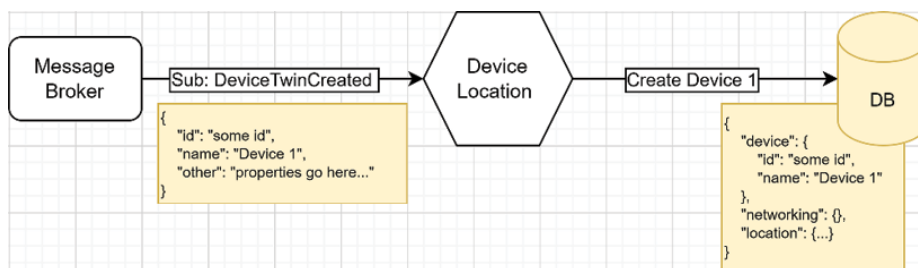


Figure 16.11: The DeviceLocation microservice replaying the DeviceTwinCreated event to create its materialized view of the device twin

2. The `DeviceLocation` microservice receives the `NetworkingInfoUpdated` event, which updates the networking type to `MQTT`, leading to the following:

```
{
  "device": {
    "id": "some id",
    "name": "Device 1"
  },
  "networking": {
    "type": "MQTT"
  },
  "location": {...}
}
```

This is demonstrated by the following diagram:

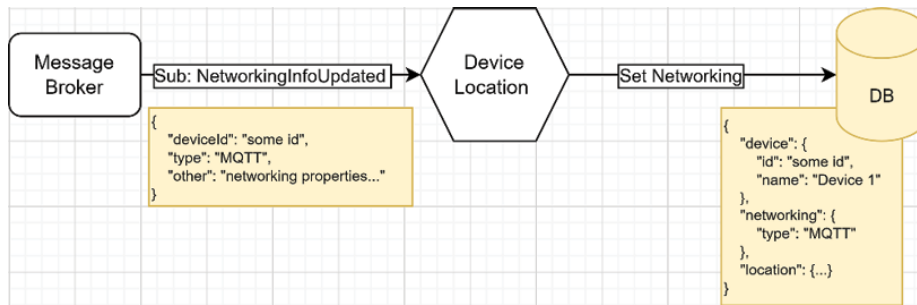


Figure 16.12: The DeviceLocation microservice replaying the NetworkingInfoUpdated event to update its materialized view of the device twin

3. The DeviceLocation microservice receives the DeviceTwinUpdated event, updating the device's name. The final model looks like this:

```
{
  "device": {
    "id": "some id",
    "name": "Kitchen Thermostat"
  },
  "networking": {
    "type": "MQTT"
  },
  "location": {...}
}
```

This is demonstrated by the following diagram:

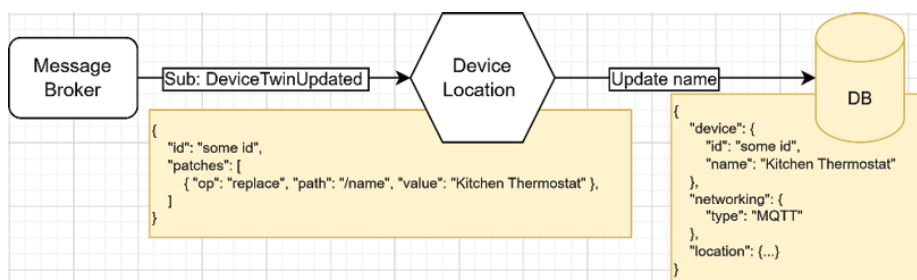


Figure 16.13: The DeviceLocation microservice replaying the DeviceTwinUpdated event to update its materialized view of the device twin

From there, the DeviceLocation microservice is initialized and ready. A user could set the kitchen thermostat's location on the map or continue to play with the other parts of the system. When a user queries the DeviceLocation microservice for information about Kitchen Ther-

`mostat`, it displays the **materialized view**, which contains all the required information without sending external requests.

With that in mind, we could spawn new instances of the `DeviceLocation` microservice or other microservices, and they could generate their materialized views from past events—all of that with very limited to no knowledge of other microservices. In this type of architecture, a microservice can only know about events, not the other microservices. How a microservice handles events should be relevant only to that microservice, never to the others. The same applies to both publishers and subscribers.

This example illustrates the event sourcing pattern, integration events, the materialized view, the use of a message broker, and the Publish-Subscribe pattern.

In contrast, using direct communication (HTTP, gRPC, and so on) would look like this:

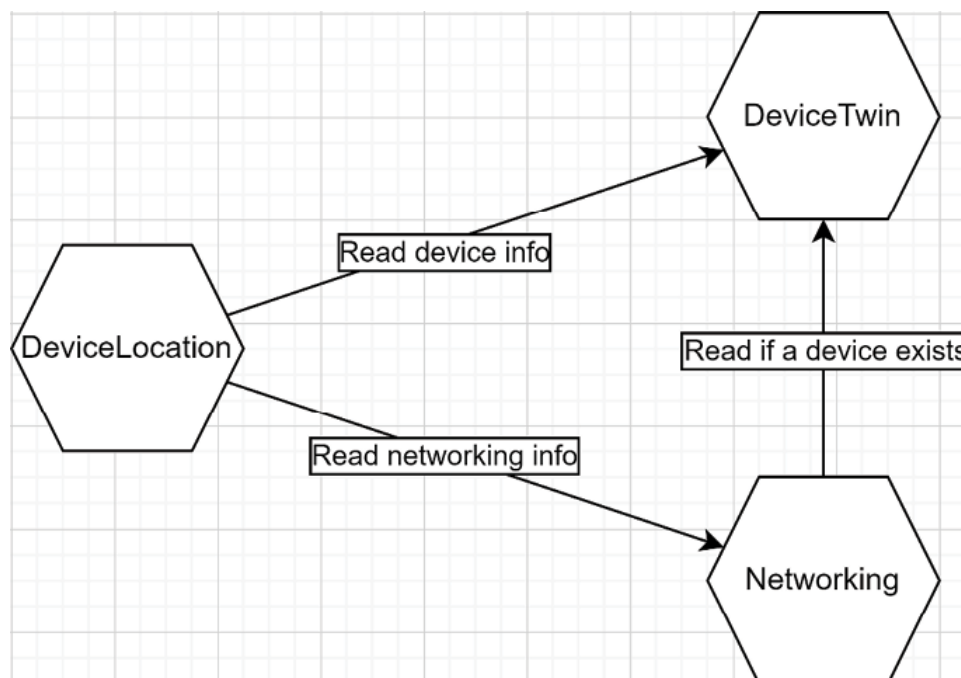


Figure 16.14: Three microservices communicating directly with one another

If we compare both approaches, by looking at the first diagram (*Figure 16.7*), we can see that the message broker plays the role of a **mediator** and breaks the direct coupling between the microservices. By looking at the preceding diagram (*Figure 16.14*), we can see the tight coupling between the microservices, where the `DeviceLocation` microservice would need to interact with the `DeviceTwin` and `Networking` microservices directly to build the equivalent of its materialized view. Fur-

thermore, the `DeviceLocation` microservice translates one interaction into three since the `Networking` microservice also talks to the `DeviceTwin` microservice, leading to indirect tight coupling between microservices, which can negatively impact performance.

Suppose eventual consistency is not an option, or the Publish-Subscribe pattern cannot be applied or could be too hard to apply to your scenario. In this case, microservices can directly call each other. They can achieve this using HTTP, gRPC, or any other means that best suit that particular system's needs.

I won't be covering this topic in this book, but one thing to be careful of when calling microservices directly is the indirect call chain that could bubble up fast. You don't want your microservices to create a super deep call chain, or your system will most likely become very slow, very fast. Here is an abstract example of what could happen to illustrate what I mean. A diagram is often better than words:

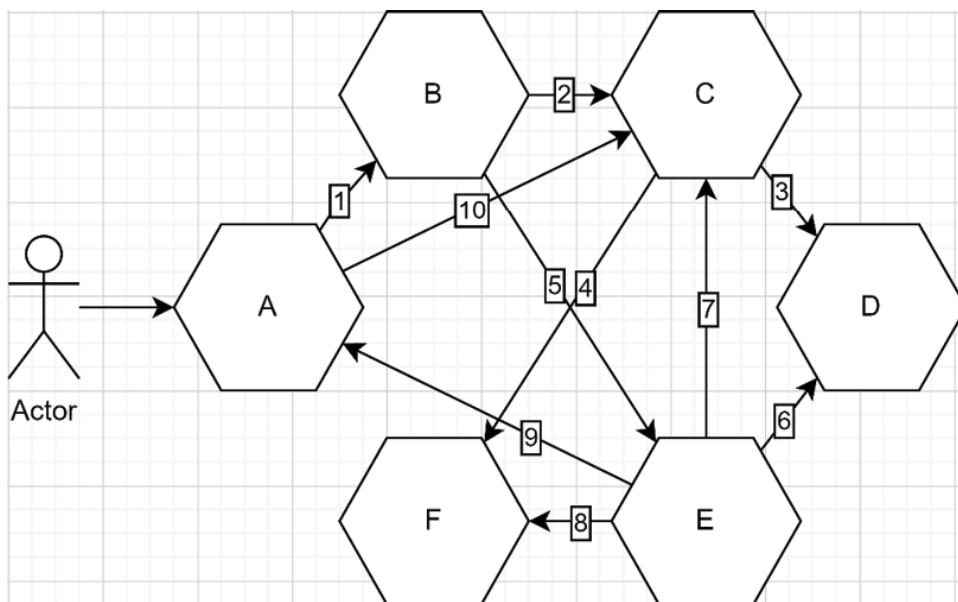


Figure 16.15: A user calling microservice A, which then triggers a chain reaction of subsequent calls, leading to disastrous performance

In terms of the preceding diagram, let's think about failures (for one). If microservice C goes offline, the whole request ends with an error. No matter the measures we put in place to mitigate the risks, if microservice C cannot recover, the system will remain down; goodbye to microservices' promise of independence. Another issue is latency: ten calls are made for a single operation; that takes time.

Such chatty systems have most likely emerged from an incorrect domain modeling phase, leading to multiple microservices working together to

handle trivial tasks. Now think of *Figure 16.15* but with 500 microservices instead of 6. That could be catastrophic!

This type of interdependent microservices system is known as the **Death Star anti-pattern**. Personally, I see the Death Star anti-pattern as a *distributed big ball of mud*. One way to avoid such pitfalls is to ensure the bounded contexts are well segregated and that responsibilities are well distributed.

A good domain model should allow you to avoid building a Death Star and create the “most correct” system possible instead. No matter the type of architecture you choose, if you are not building the right thing, you may well end up with a big ball of mud or a Death Star. Of course, the Pub-Sub pattern can help us break the tight coupling between microservices to avoid such issues.

Conclusion

The Publish-Subscribe pattern uses events to break tight coupling between parts of an application. In a microservices architecture, we can use a message broker and integration events to allow microservices to talk to each other indirectly. The different pieces are now coupled with the data contract representing the event (its schema) instead of each other, leading to a potential gain in flexibility. One risk of this type of architecture is breaking events' consumers by publishing breaking changes in the event's format without letting consumers know or without having events versioning in place so consumers can self-manage themselves. Therefore, it is critical to think about event schema evolutions thoroughly. Most systems evolve, as will events, but since schemas are the glue between systems in a Publish-Subscribe model, it is essential to treat them as such. Some brokers, like Apache Kafka, offer a schema store and other mechanisms to help with these; some don't.

Then, we can leverage the event sourcing pattern to persist those events, allowing new microservices to populate their databases by replaying past events. The event store then becomes the source of truth of those systems. Event sourcing can also become very handy for tracing and auditing purposes since the whole history is persisted. We can also replay messages to recreate the system's state at any given point in time, making it very powerful for debugging purposes. The storage size requirement for the event store is something to consider before going down the event sourcing path. The event store could grow quite large because we keep all messages since the beginning of time and could grow fast based on the quantity of events sent. You could compact the history to reduce the data size but lose

part of the history. Once again, you must decide based on the requirements and ask yourself the appropriate questions. For example, is it acceptable to lose part of the history? How long should we keep the data? Do we want to keep the original data in cheaper storage if we need it later? Do we even need replaying capabilities? Can we afford to keep all the data forever? Craft your list of questions based on the specific business problem you want to solve. This advice applies to all aspects of software engineering: clearly define the business problem first, then find how to fix it.

Such patterns can be compelling but can also take time to learn and implement. Like with message queues, cloud providers offer fully managed brokers as a service. Those can be faster to get started with than building and maintaining your own infrastructure. If building servers is your thing, you can use open source software to “economically” build your stack or just pay for managed instances of such software to save yourself the trouble. The same tips as with message queues apply here; for example, you can leverage a managed service for your production environment and a local version on the developer’s machine.

Apache Kafka is one of the most popular event brokers that enables advanced functionalities like event streaming. Kafka has partially and fully managed cloud offerings like Confluent Cloud. Redis Pub/Sub is another open source project that has fully managed cloud offerings. Redis is also a key-value store trendy for distributed caching scenarios. Other offerings are (but are not limited to) Solace PubSub+, RabbitMQ, and ActiveMQ. Once again, I suggest comparing the offerings with your requirements to make the best choice for your scenarios.

Now, let’s see how the Publish-Subscribe pattern can help us follow the **SOLID** principles at cloud-scale:

- **S:** Helps centralize and divide responsibilities between applications or components without them directly knowing each other, breaking tight coupling.
- **O:** Allows us to change how publishers and subscribers behave without directly impacting the other microservices (breaking tight coupling between them).
- **L:** N/A
- **I:** Each event can be as small as needed, leading to multiple smaller communication interfaces (data contracts).
- **D:** The microservices depend on events (abstractions) instead of concretions (the other microservices), breaking tight coupling between them and inverting the dependency flow.

As you may have noticed, pub-sub is very similar to message queues. The main difference is the way messages are read and dispatched:

- Queues: messages are pulled one at a time, consumed by one service, then disappear.
- Pub-Sub: messages are also read in order and are sent to all consumers instead of to only one, like with queues.

Observer design pattern

I intentionally kept the Observer pattern out of this book since we rarely need it in .NET. C# offers multicast events, which are well versed in replacing the Observer pattern (in most cases). If you don't know the Observer pattern, don't worry—chances are, you will never need it anyway. Nevertheless, if you already know the Observer pattern, here are the differences between it and the Pub-Sub pattern.

In the Observer pattern, the subject keeps a list of its observers, creating direct knowledge of their existence. Concrete observers also often know about the subject, which leads to even more knowledge of other entities, leading to more coupling.

In the Pub-Sub pattern, the publisher is not aware of the subscribers; it is only aware of the message broker. The subscribers are not aware of the publishers either, only of the message broker. The publishers and subscribers are linked only through the data contract of the messages they are either publishing or receiving.

We could view the Pub-Sub pattern as the distributed evolution of the Observer pattern or more precisely, like adding a mediator to the Observer pattern.

Next, we explore some patterns that directly call other microservices by visiting a new kind of **Façade**: the **Gateway**.

Introducing Gateway patterns

When building a microservices-oriented system, the number of services grows with the number of features; the bigger the system, the more microservices you have.

When you think about a user interface that has to interact with such a system, this can become tedious, complex, and inefficient (dev-wise and speed-wise). Gateways can help us achieve the following:

- Hide complexity by routing requests to the appropriate services.
- Hide complexity by aggregating responses, translating one external request into many internal ones.
- Hide complexity by exposing only the subset of features that a client needs.
- Translate an external request into another protocol that's used internally.

A gateway can also centralize different processes, such as logging and caching requests, authenticating and authorizing users and clients, enforcing request rate limits, and other similar policies.

You can see gateways as façades, but instead of being a class in a program, it is a program of its own, shielding other programs. There are multiple variants of the Gateway pattern, and we explore many of them here.

Regardless of the type of gateway you need, you can code it yourself or leverage existing tools to speed up the development process.

Tip

Beware that there is a strong chance that your homemade gateway version 1.0 has more flaws than a proven solution. This tip is not only applicable to gateways but to most complex systems. That being said, sometimes, there is no proven solution that does exactly what we want, and we have to code it ourselves, which is where the real fun begins!

An open source project that could help you out is Ocelot (<https://adpg.link/UwiY>). It is an application gateway written in .NET Core that supports many things that we expect from a gateway. You can route requests using configuration or write custom code to create advanced routing rules. Since it is open source, you can contribute to it, fork it, and explore the source code if you need to.

A gateway is a **reverse proxy** that fetches the information that's been requested by a client. That information can come from one or more resources, possibly located on one or more servers. Microsoft is working on a reverse proxy named YARP, which is also open source (<https://adpg.link/YARP>). Microsoft claims they are building it for

their internal teams, so it will most likely evolve and be maintained over time (my guess).

Now, let's explore a few types of gateways.

Gateway Routing pattern

We can use this pattern to hide the complexity of our system by having the gateway route requests to the appropriate services.

For example, let's say that we have two microservices: one that holds our device data and another that manages device locations. We want to show the latest known location of a specific device (`id=102`) and display its name and model.

To achieve that, a user requests the web page, and then the web page calls two services (see the following diagram). The `DeviceTwin` microservice is accessible from `service1.domain.com`, and the `Location` microservice is accessible from `service2.domain.com`. From there, the web application has to keep track of what services use what domain name. The UI has to handle more complexity as we add more microservices. Moreover, if at some point we decide to change `service1` to `device-twins` and `service2` to `location`, we'd need to update the web application as well. If there is only a UI, it is still not so bad, but if you have multiple user interfaces, that means each of them has to handle that complexity.

Furthermore, if we want to hide the microservices inside a private network, it would be impossible unless all the user interfaces are also part of that private network (which exposes it):

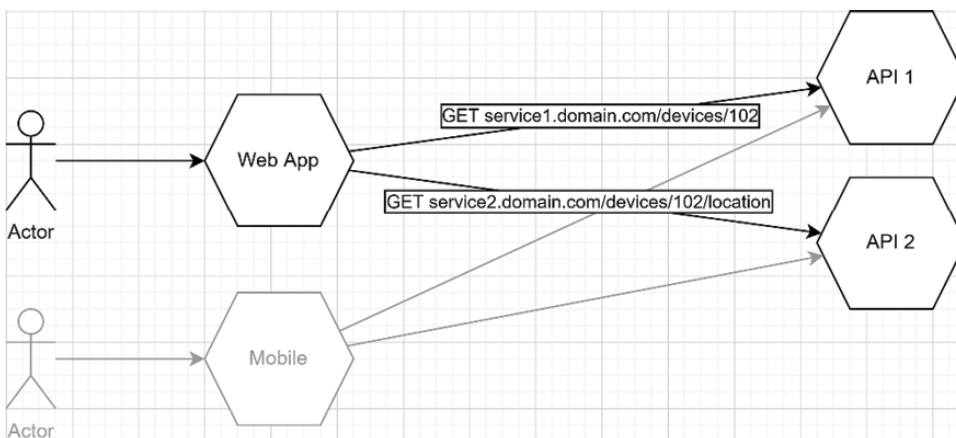


Figure 16.16: A web application and a mobile app that are calling two microservices directly

To fix some of these issues, we can implement a gateway that does the routing for us. That way, instead of knowing what services are accessible through what sub-domain, the UI only has to know the gateway:

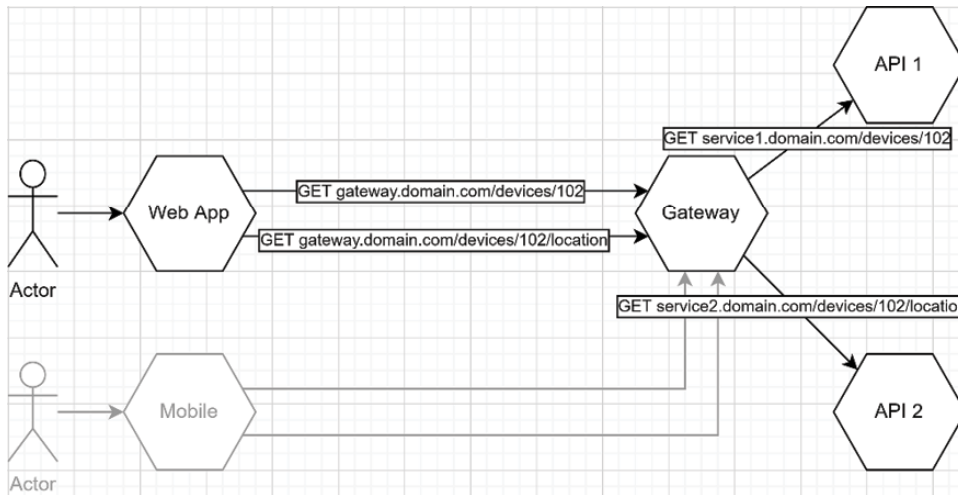


Figure 16.17: A web application and a mobile app that are calling two microservices through a gateway application

Of course, this brings some possible issues to the table as the gateway becomes a single point of failure. You could consider using a load balancer to ensure that you have strong enough availability and fast enough performance. Since all requests pass through the gateway, you may need to scale it up at some point.

You should also ensure the gateway supports failure by implementing different resiliency patterns, such as **Retry** and **Circuit Breaker**. The chances that an error will occur on the other side of the gateway increase with the number of microservices you deploy and the number of requests sent to those microservices.

You can also use a routing gateway to reroute the URI to create easier-to-use URI patterns. You can also reroute ports; add, update, or remove HTTP headers; and more. Let's explore the same example but using different URIs. Let's assume the following:

Microservice	URI
API 1 (get a device)	<code>internal.domain.com:8001/{id}</code>
API 2 (get a device location)	<code>internal.domain.com:8002/{id}</code>

UI developers would have a harder time remembering what port is leading to what microservice and what is doing what (and who could blame them?). Moreover, we could not transfer the requests as we did earlier (only routing the domain). We could use the gateway as a way to create memorable URI patterns for developers to consume, like these:

Gateway URI	Microservice URI
<code>gateway.domain.com/devices/{id}</code>	<code>internal.domain.com:8001/{id}</code>
<code>gateway.domain.com/devices/{id}/location</code>	<code>internal.domain.com:8002/{id}</code>

As you can see, we took the ports out of the equation to create usable, meaningful, and easy-to-remember URIs.

However, we are still making two requests to the gateway to display one piece of information (the location of a device and its name/model), which leads us to our next Gateway pattern.

Gateway Aggregation pattern

Another role that we can give to a gateway is to aggregate requests to hide complexity from its consumers. Aggregating multiple requests into one makes it easier for consumers of a microservices system to interact with it; clients need to know about one endpoint instead of multiple. Moreover, it moves the chattiness from the client to the gateway, which is closer to the microservices, lowering the many calls' latency, thus making the request-response cycle faster.

Continuing with our previous example, we have two UI applications that contain a feature to show a device's location on a map before identifying it using its name/model. To achieve this, they must call the device twin endpoint to obtain the device's name and model, as well as the location endpoint to get its last known location. So, two requests to display a small box, times two UIs, means four requests to maintain for a simple feature. If we extrapolate, we could end up managing a near-endless number of HTTP requests for a handful of features.

Here is a diagram showing our feature in its current state:

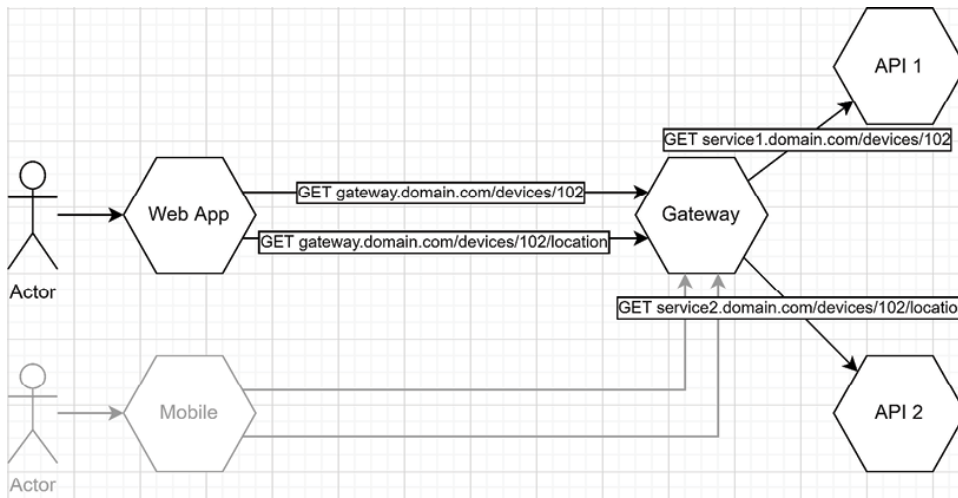


Figure 16.18: A web application and a mobile app that are calling two microservices through a gateway application

To remedy this problem, we can apply the Gateway Aggregation pattern to simplify our UIs and offload the responsibility of managing those details to the gateway.

By applying the Gateway Aggregation pattern, we end up with the following simplified flow:

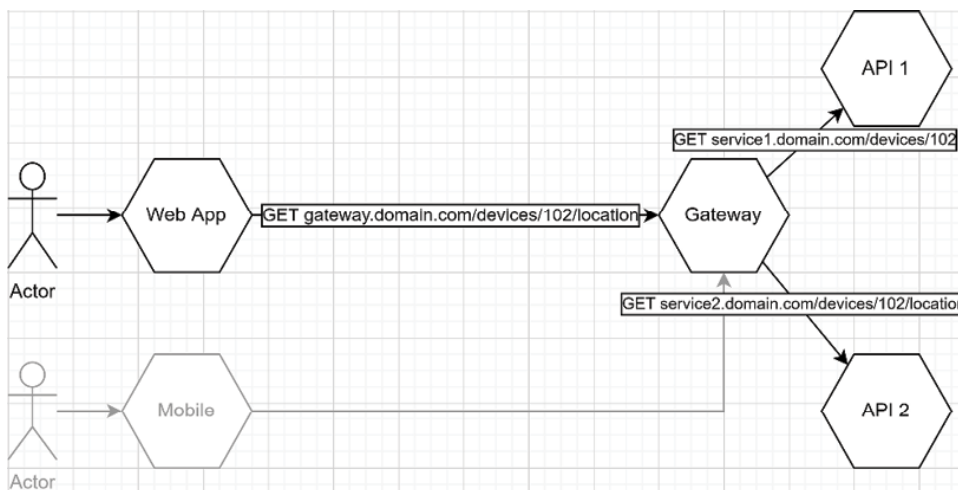


Figure 16.19: A gateway that aggregates the response of two requests to serve a single request from both a web application and a mobile app

In the previous flow, the Web App calls the Gateway that calls the two APIs, then crafts a response combining the two responses it got from the APIs. The Gateway then returns that response to the Web App. With that in place, the Web App is loosely coupled with the two APIs, with the Gateway playing the middleman. With only one HTTP request, the Web App has all the information it needs, aggregated by the Gateway.

Next, let's explore the steps that occurred. In the following diagram, we can see the Web App makes a single request (1), while the gateway makes two calls (2 and 4). In the diagram, the requests are sent in series, but we could have sent them in parallel to speed things up:

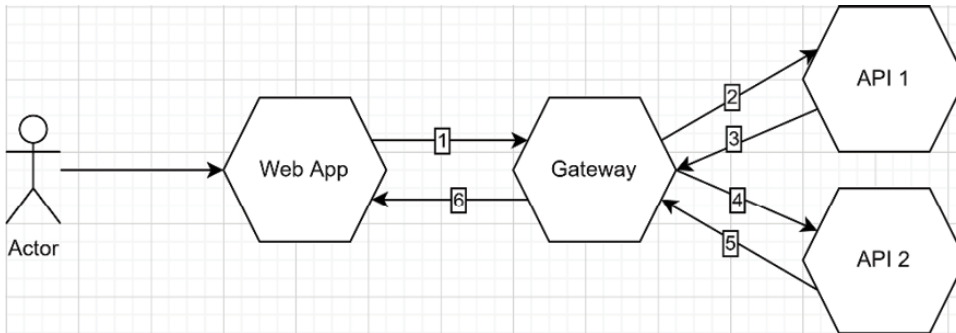


Figure 16.20: The order in which the requests take place

Like the routing gateway, an aggregation gateway can become the bottleneck of your application and a single point of failure, so beware of that.

Another important point to note is the latency between the gateway and the internal APIs. If the latency is too high, your clients are going to wait for every response. So, deploying the gateway close to the microservices it interacts with could become crucial for system performance. The gateway can also implement caching to improve performance so that subsequent requests are faster.

Next, we explore another type of gateway that creates specialized gateways instead of generic ones.

Backends for Frontends pattern

The Backends for Frontends pattern is yet another variation of the Gateway pattern. With Backends for Frontends, instead of building a general-purpose gateway, we build a gateway per user interface (or for an application that interacts with your system), lowering complexity. Moreover, it allows for fine-grained control of what endpoints are exposed. It removes the chances of app B breaking when changes are made to app A. Many optimizations can come out of this pattern, such as sending only the data that's required for each call instead of sending data that only a few applications are using, saving some bandwidth along the way.

Let's say that our Web App needs to display more data about a device. To achieve that, we would need to change the endpoint and send that extra information to the mobile app as well.

However, the mobile app doesn't need that information since it doesn't have room on its screen to display it. Here is an updated diagram that replaces the single gateway with two gateways, one per frontend.

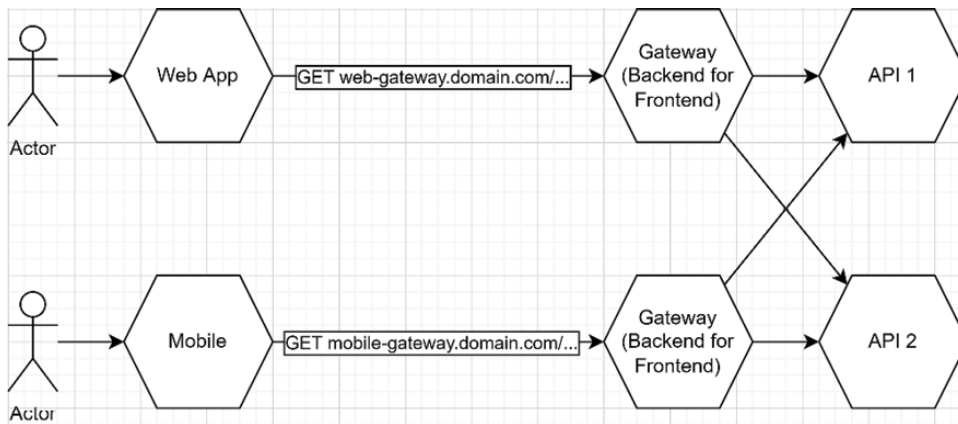


Figure 16.21: Two backends for frontends gateways; one for the Web App and one for the Mobile App

By doing this, we can now develop specific features for each frontend without impacting the other. Each gateway is now shielding its particular frontend from the rest of the system and the other frontend. This is the most important benefit this pattern brings to the table: independence between clients.

Once again, the Backends for Frontends pattern is a gateway. And like other variations of the Gateway pattern, it can become the bottleneck of its frontend and its single point of failure. The good news is that the outage of one backend for frontend gateway limits the impact to a single frontend, shielding the other frontends from that downtime.

Mixing and matching gateways

Now that we've explored three variations of the Gateway pattern, it is important to note that we can mix and match them, either at the codebase level or as multiple microservices.

For example, a gateway can be built for a single client (backend for frontend), perform simple routing, and aggregate results.

We can also mix them as different applications, for example, by putting multiple backend for frontend gateways in front of a more generic gateway to simplify the development and maintenance of those backends for frontends.

Beware that each hop has a cost. The more pieces you add between your clients and your microservices, the more time it will take for those clients to receive the response (latency). Of course, you can put mechanisms in place to lower that overhead, such as caching or non-HTTP protocols such as gRPC, but you still must consider it. That goes for everything, not just gateways.

Here is an example illustrating this:

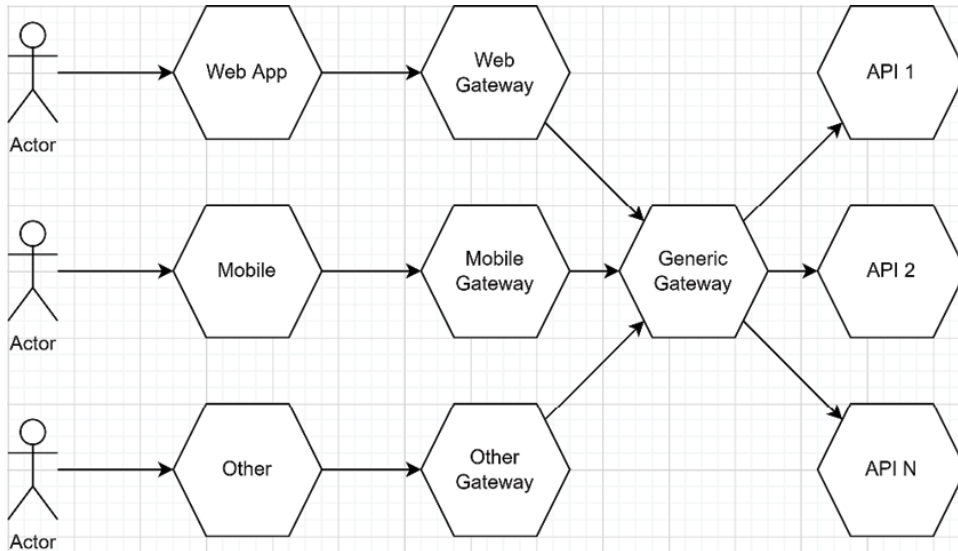


Figure 16.22: A mix of the Gateway patterns

As you’ve possibly guessed, the Generic Gateway is the single point of failure of all applications, while at the same time, each backend for frontend gateway is a point of failure for its specific client.

Service mesh

A service mesh is an alternative to help microservices communicate with one another. It is a layer, outside of the application, that proxies communications between services. Those proxies are injected on top of each service and are referred to as **sidecars**. The service mesh can also help with distributed tracing, instrumentation, and system resiliency. If your system needs service-to-service communication, a service mesh would be an excellent place to look.

Conclusion

A gateway is a façade or reverse proxy that shields or simplifies access to one or more other services. In this section, we explored the following:

- **Routing:** This forwards a request from point A to point B.
- **Aggregation:** This combines the result of multiple sub-requests into a single response.
- **Backends for Frontends:** This is used in a one-to-one relationship with a frontend.

We can use any microservices pattern, including gateways, and like any other pattern, we can mix and match them. Just consider the advantages, but also the drawbacks, that they bring to the table. If you can live with them, well, you've got your solution.

Gateways often end up being the single point of failure, so that is a point to consider. On the other hand, a gateway can have multiple instances running simultaneously behind a load-balancer (see *Appendix B* for more information about scaling). Moreover, we must also consider the delay that's added by calling a service that calls another service since that slows down the response time.

All in all, a gateway is a great tool to simplify consuming microservices. They also allow hiding the microservices topology behind them, possibly even isolated in a private network. They can also handle cross-cutting concerns such as security.

Note

It is imperative to use gateways as a requests passthrough and avoid coding business logic into them; gateways are just reverse proxies. Think single responsibility principle: a gateway is a façade in front of your microservices cluster. Of course, you can unload specific tasks into your gateways like authorization, resiliency (retry policies, for example), and similar cross-cutting concerns, but the business logic must remain in the backend microservices.

I strongly recommend against rolling out your own gateway and suggest leveraging existing offerings instead. This is why I chose not to add C# code in this section. There are many open source and cloud gateways that you can use in your application. Using existing components leaves you more time to implement the business rules that solve the issues your program is trying to tackle.

Of course, cloud-based offerings exist, like Azure Application Gateway and Amazon API Gateway. Both are extendable with cloud offerings like load-balancers and **web application firewalls (WAF)**. For example,

Azure Application Gateway also supports autoscaling, zone redundancy, and can serve as **Azure Kubernetes Service (AKS)** Ingress Controller (in a nutshell, it controls the traffic to your microservices cluster). For more information about Kubernetes and containers, see *Appendix B*.

If you want more control over your gateways or to deploy them with your application, you can leverage one of the options that are out there. For simplicity, I picked two to talk to you about: Ocelot and Envoy.

Ocelot is an open source production-ready API Gateway programmed in .NET. Ocelot supports routing, request aggregation, load-balancing, authentication, authorization, rate limiting, and more. It also integrates well with Identity Server, an **OpenID Connect (OIDC)** and OAuth 2.0 implementation, written in .NET. The biggest advantage of Ocelot, in my eyes, is the fact that you create the .NET project yourself, install a NuGet package, configure your gateway, then deploy it like you would any other ASP.NET Core application. Since Ocelot is written in .NET, it is easier to extend it if needed or contribute to the project or its ecosystem by sharing your improvements.

Envoy is an “*open source edge and service proxy, designed for cloud-native applications*,” to quote their website. Envoy is a **Cloud Native Computing Foundation (CNCF)** graduated project, originally created by Lyft. Envoy was designed to run as a separate process from your application, allowing it to work with any programming language. Envoy can serve as a gateway and has an extendable design through TCP/UDP and HTTP filters, supports HTTP/2 and HTTP/3, gRPC, and more.

Which offering to choose? If you are looking for a fully managed service, look at the cloud provider’s offering of your choice. Consider Ocelot if you are looking for a configurable HTTP gateway that supports the patterns covered in this chapter. If you have complex use cases that Ocelot does not support, you can look into Envoy, a proven offering with many advanced capabilities. Please keep in mind that these are just a few possibilities that can play the role of a gateway in a microservices architecture system and are not intended to be a complete list.

Now, let’s see how gateways can help us follow the **SOLID** principles at cloud-scale:

- **S:** A gateway can handle routing, aggregation, and other similar logic that would otherwise be implemented in different components or applications.
- **O:** I see many ways to attack this one, but here are two takes on this:

1. Externally, a gateway could reroute its sub-requests to new URIs without its consumers knowing about it, as long as its contract does not change.
 2. Internally, a gateway could load its rules from configurations, allowing it to change without updating its code (this one would be an implementation detail).
- **L:** We could see the previous point (b) as *not changing the correctness of the application*.
 - **I:** Since a backend for frontend gateway serves a single frontend system, that means one contract (interface) per frontend system, leading to multiple smaller interfaces instead of one big general-purpose gateway.
 - **D:** We could see a gateway as an abstraction, hiding the real microservices (implementations) and inverting the dependency flow.

Next, we revisit CQRS on a distributed scale.

Revisiting the CQRS pattern

Command Query Responsibility Segregation (CQRS), explored in *Chapter 14, Mediator and CQRS Design Patterns*, applies the **Command Query Separation (CQS)** principle. Compared to what we saw in *Chapter 14, Mediator and CQRS Design Patterns*, we can push CQRS further using microservices or serverless computing. Instead of simply creating a clear separation between commands and queries, we can divide them even more by using multiple microservices and data sources.

CQS is a principle stating that a method should either return data or mutate data, but not both. On the other hand, **CQRS** suggests using one model to read the data and one model to mutate the data.

Serverless computing is a cloud execution model where the cloud provider manages the servers and allocates the resources on-demand, based on usage. Serverless resources fall into the platform as a service (PaaS) offering.

Let's use IoT again as an example; we queried the last known location of a device in the previous examples, but what about the device updating that location? This can mean pushing many updates every minute. To solve this issue, we are going to use CQRS and focus on two operations:

- Updating the device location.
- Reading the last known location of a device.

Simply put, we have a Read Location microservice, a Write Location microservice, and two databases. Remember that each microservice should own its data. That way, a user can access the last known device location through the read microservice (query model), while a device can punctually send its current position to the write microservice (command model). By doing that, we split the load from reading and writing the data as both occur at different frequencies:

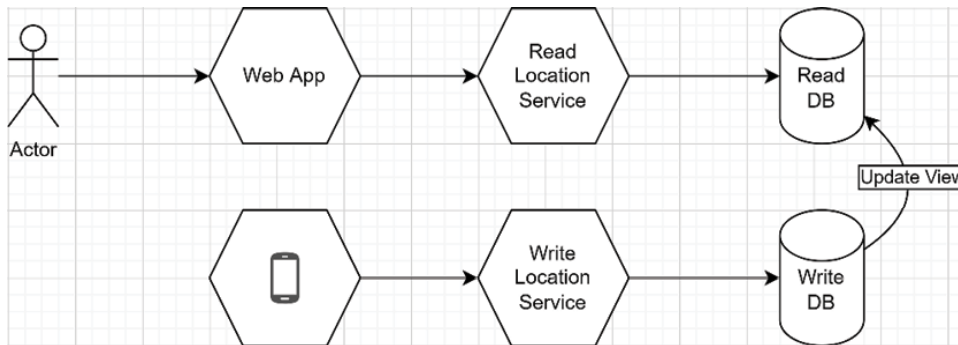


Figure 16.23: Microservices that apply CQRS to divide the reads and writes of a device's location

In the preceding schema that illustrates the concept, the reads are queries, and the writes are commands. How to update the Read DB once a new value is added to the Write DB depends on the technology at play. One essential thing in this type of architecture is that, per the CQRS pattern, a command should not return a value, enabling a “fire and forget” scenario. With that rule in place, consumers don’t have to wait for the command to complete before doing something else.

Note

Fire and forget does not apply to every scenario; sometimes, we need synchronization. Implementing the Saga pattern is one way to solve coordination issues.

Conceptually, we can implement this example by leveraging serverless cloud infrastructures, such as Azure Functions and Table Storage. Let’s revisit this example using those components:

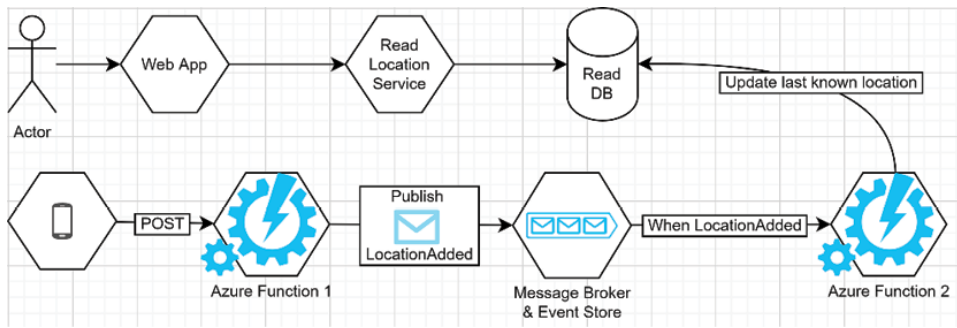


Figure 16.24: Using Azure services to manage a CQRS implementation

The previous diagram illustrates the following:

1. The device sends its location at a regular interval by posting it to Azure Function 1.
2. Azure Function 1 then publishes the `LocationAdded` event to the event broker, which is also an event store (the Write DB).
3. All subscribers to the `LocationAdded` event can now handle the event appropriately; in this case, Azure Function 2.
4. Azure Function 2 updates the last known location of the device in the Read DB.
5. Any subsequent queries should result in getting the new location.

The message broker is also the event store in the preceding diagram, but we could store events elsewhere, such as in an Azure Storage Table, in a time-series database, or in an Apache Kafka cluster. Azure-wise, the datastore could also be CosmosDB. Moreover, I abstracted this component for multiple reasons, including the fact that there are multiple “as-a-service” offerings to publish events in Azure, and there are multiple ways of using third-party components as well (both open source and proprietary).

Furthermore, the example demonstrates **eventual consistency** well. All the last known location reads between *steps 1* and *4* get the old value while the system processes the new location updates (commands). If the command processing slows down for some reason, a longer delay could occur before the next read database updates. The commands could also be processed in batches, leading to another kind of delay. No matter what happens with the command processing, the read database would be available all that time whether it has the latest data or not and whether the write system is overloaded or not. This is the beauty of this type of design, but it is more complex to implement and maintain.

Time-series databases are optimized for temporally querying and storing data, where you always append new records without updating old ones. This kind of NoSQL database can be useful for temporal-intensive usage.

Once again, we used the Publish-Subscribe pattern to get another scenario going. Assuming that events are persisted forever, the previous example could also support event sourcing. Furthermore, new services could subscribe to the `LocationAdded` event without impacting the code that has already been deployed. For example, we could create a SignalR microservice that pushes the updates to its clients. It is not CQRS-related, but it flows well with everything that we've explored so far, so here is an updated conceptual diagram:

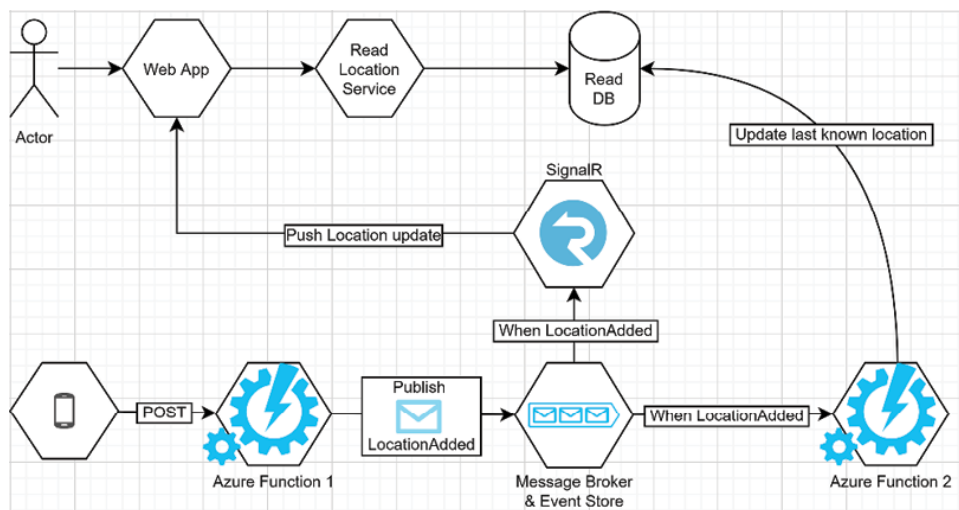


Figure 16.25: Adding a SignalR service as a new subscriber without impacting the other part of the system

The SignalR microservice could be custom code or an Azure SignalR Service (backed by another Azure Function); it doesn't matter. Here I wanted to illustrate that it is easier to drop new services into the mix when using a Pub-Sub model than with point-to-point communication.

As you can see, a microservices system adds more and more small pieces that indirectly interconnect with each other over one or more message brokers. Maintaining, diagnosing, and debugging such systems is harder than with a single application; that's the **operational complexity** we talked about earlier. However, containers can help deploy and maintain such systems; see *Appendix B* for more information about containers.

Starting in ASP.NET Core 3.0, the ASP.NET Core team invested much effort into **distributed tracing**. Distributed tracing is necessary to find failures

and bottlenecks related to an event that flows from one program to another (such as microservices). If something bugs out, it is important to trace what the user did to isolate the error, reproduce it, and then fix it. The more independent pieces there are, the harder it can become to make that trace possible. This is outside the scope of this book, but it is something to consider before jumping into the microservices adventure.

Conclusion

CQRS helps divide queries and commands and helps encapsulate and isolate each block of logic independently. Mixing that concept with serverless computing or microservices architecture allows us to scale reads and writes independently. We can also use different databases, empowering us with the tools we need for the transfer rate required by each part of that system (for example, frequent writes and occasional reads or vice versa).

Major cloud providers like Azure and AWS provide serverless offerings to help support such scenarios. Each cloud provider's documentation should help you get started. Meanwhile, for Azure, we have Azure Functions, Event Grid, Event Hubs, Service Bus, Cosmos DB, and more. Azure also offers bindings between the different services that are triggered or react to events for you, removing a part of the complexity.

Now, let's see how CQRS can help us follow the **SOLID** principles at the cloud scale:

- **S:** Dividing an application into smaller reads and writes applications (or functions) leans toward encapsulating single responsibilities into different programs.
- **O:** CQRS, mixed with serverless computing or microservices, helps extend the software without the need for us to modify the existing code by adding, removing, or replacing applications.
- **L:** N/A
- **I:** CQRS set us up to create multiple small interfaces (or programs) with a clear distinction between commands and queries.
- **D:** N/A

Exploring the Microservice Adapter pattern

The Microservice Adapter pattern allows adding missing features, adapting one system to another, or migrating an existing application to an event-driven architecture model, to name a few possibilities. The Mi-

crosservice Adapter pattern is similar to the Adapter pattern we cover in *Chapter 9, Structural Patterns* but applied to a microservices system that uses event-driven architecture instead of creating a class to adapt an object to another signature.

In the scenarios we cover in this section, the microservices system represented by the following diagram can be replaced by a standalone application as well; this pattern applies to all sorts of programs, not just microservices, which is why I abstracted away the details:

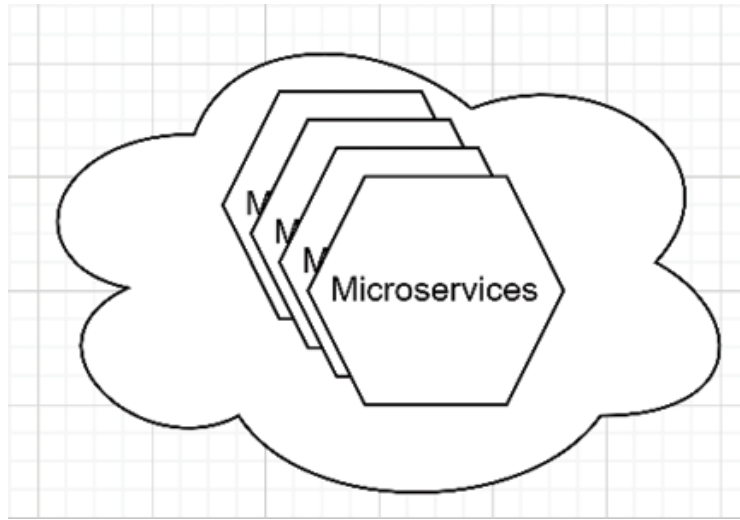


Figure 16.26: Microservice system representation used in the subsequent examples

Here are the examples we are covering next and possible usages of this pattern:

- Adapting an existing system to another.
- Decommissioning a legacy application.
- Adapting an event broker to another.

Let's start by connecting a standalone system to an event-driven one.

Adapting an existing system to another

In this scenario, we have an existing system of which we don't control the source code or don't want to change, and we have a microservices system built around an event-driven architecture model. We don't have to control the source code of the microservices system either as long as we have access to the event broker.

Here is a diagram that represents this scenario:

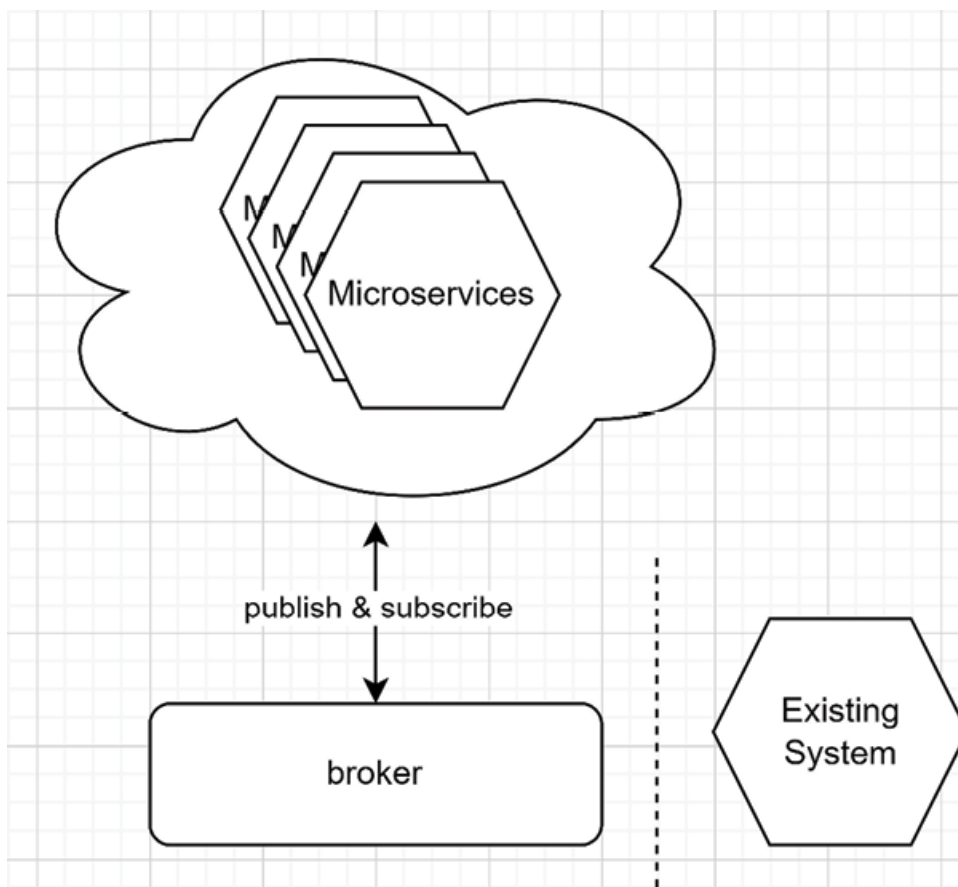


Figure 16.27: A microservices system that interacts with an event broker and an existing system that is disconnected from the microservices

As we can see from the preceding diagram, the existing system is disconnected from the microservices and the broker. To adapt the existing system to the microservices system, we must subscribe or publish certain events. In this case, let's see how to read data from the microservices (subscribe to the broker) then update that data into the existing system.

In a scenario where we control the existing system's code, we could open the source code, subscribe to one or more topics, and change the behaviors from there. In our case, we don't want to do that or can't, so we can't directly subscribe to topics, as demonstrated by the following diagram:

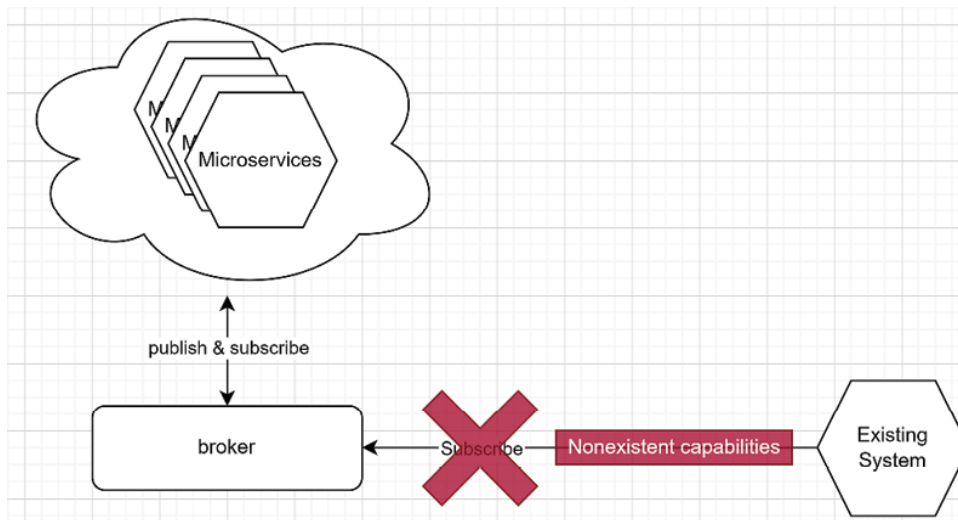


Figure 16.28: Missing capabilities to connect an existing system to an event-driven one

This is where the microservice adapter comes into play and allows us to fill the capability gap of our existing system. To add the missing link, we create a microservice that subscribes to the appropriate events, then apply the changes in the existing system, like this:

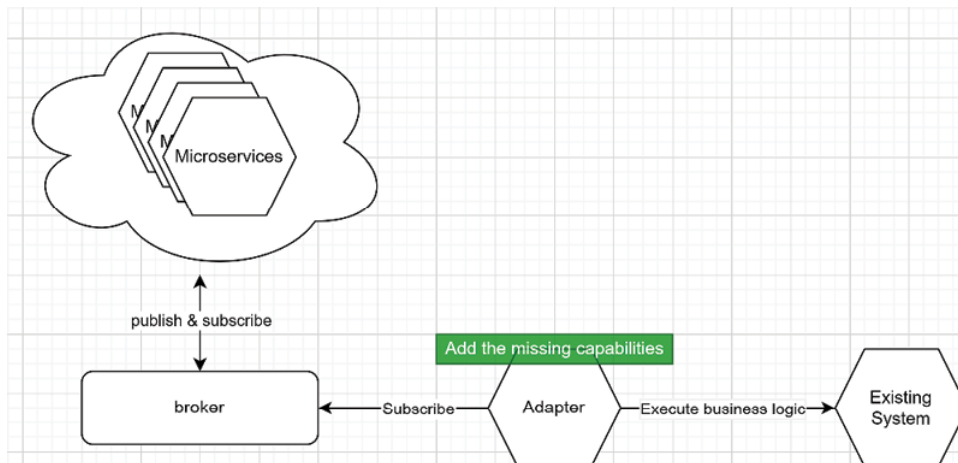


Figure 16.29: An adapter microservice adding missing capabilities to an existing system

As we can see in the preceding diagram, the `Adapter` microservice gets the events (subscribes to one or more topics) then uses that data from the microservices system to execute some business logic on the existing system.

In this design, the new `Adapter` microservice allowed us to add missing capabilities to a system we had no control over with little to no disruption to users' day-to-day activities.

The example assumes the existing system had some form of extensibility mechanism like an API. If the system does not, we would have to be more creative to interface with it.

For example, the microservices system could be an e-commerce website and the existing system a legacy inventory management system. The adapter could update the legacy system with new orders data.

The existing system could also be an old **customer relationship management (CRM)** system that you want to update when users of the microservices application execute some actions, like changing their phone number or address.

The possibilities are almost endless; you create a link between an event-driven system and an existing system you don't control or don't want to change. In this case, the microservice adapter allows us to follow the **Open-Closed principle** by extending the system without changing the existing pieces. The primary drawback is that we are deploying another microservice that has direct coupling with the existing system, which may be best for temporary solutions. On that same line of thought, next, we replace a legacy application with a new one with limited to no downtime.

Decommissioning a legacy application

In this scenario, we have a legacy application that we want to decommission and a microservices system we want to connect some existing capabilities to. To achieve this, we can create one or more adapters to migrate all features and dependencies to the new model.

Here is a representation of the current state of our system:

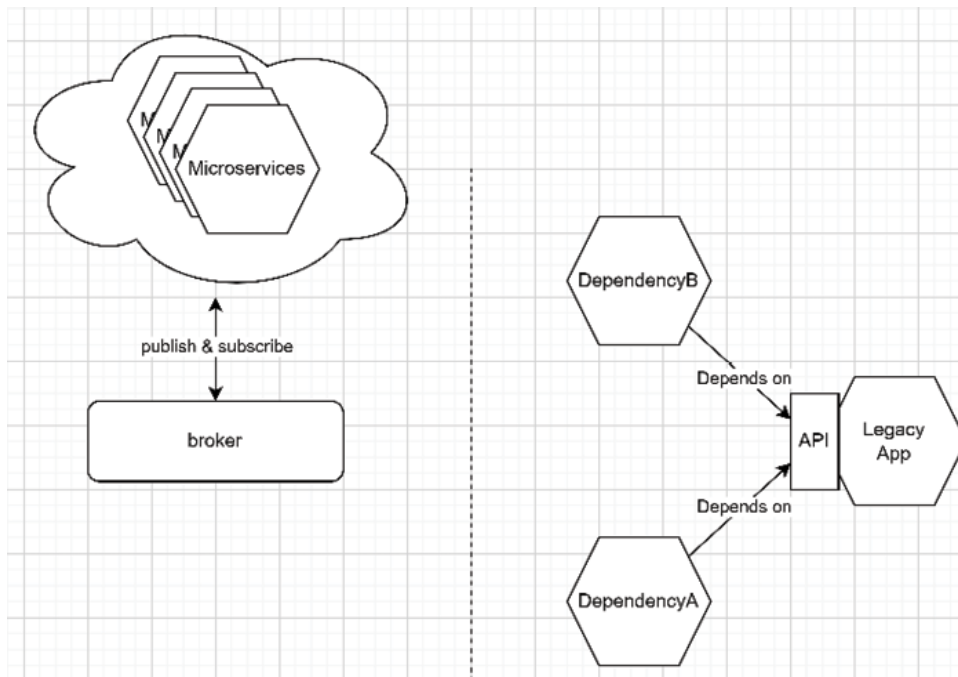


Figure 16.30: The original legacy application and its dependencies

The preceding diagram shows the two distinct systems, including the legacy application we want to decommission. Two other applications, dependency A and B, directly depend on the legacy application. The exact migration flow is strongly dependent on your use case. In the case you want to keep the dependencies, we want to migrate them first. To do that, we can create an event-driven `Adapter` microservice that breaks the tight coupling between the dependencies and the legacy application, like this:

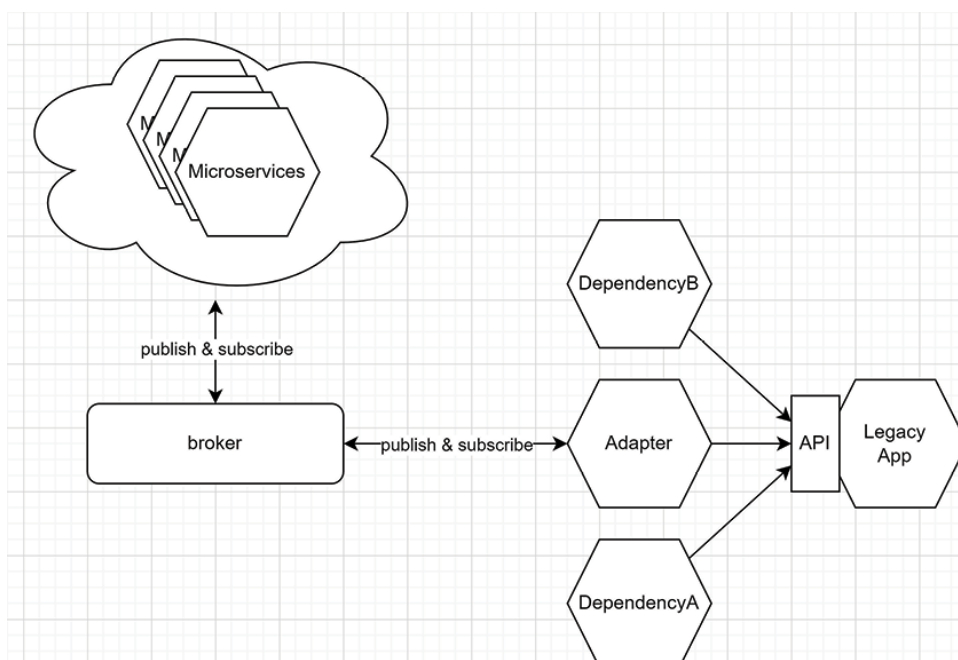


Figure 16.31: Adding a microservice adapter that implements the event-driven flow required to break tight coupling between the dependencies and the legacy application

The preceding diagram shows an `Adapter` microservice and the rest of a microservices system that communicates using an event broker. As we explored in the previous example, the adapter was placed there to connect the legacy application to the microservices. The focus of our scenario is to remove the legacy application and migrate its two dependencies. Here, we carved out the required capabilities using the adapter, allowing us to migrate the dependencies to an event-driven model and break tight-coupling with the legacy application. Such migration could be done in multiple steps, migrating each dependency one by one, and we could even create one adapter per dependency. For the sake of simplicity, I chose to draw only one adapter. If your dependencies are large or complex, you may want to revisit this choice.

Once we are done migrating the dependencies, our systems look like the following:

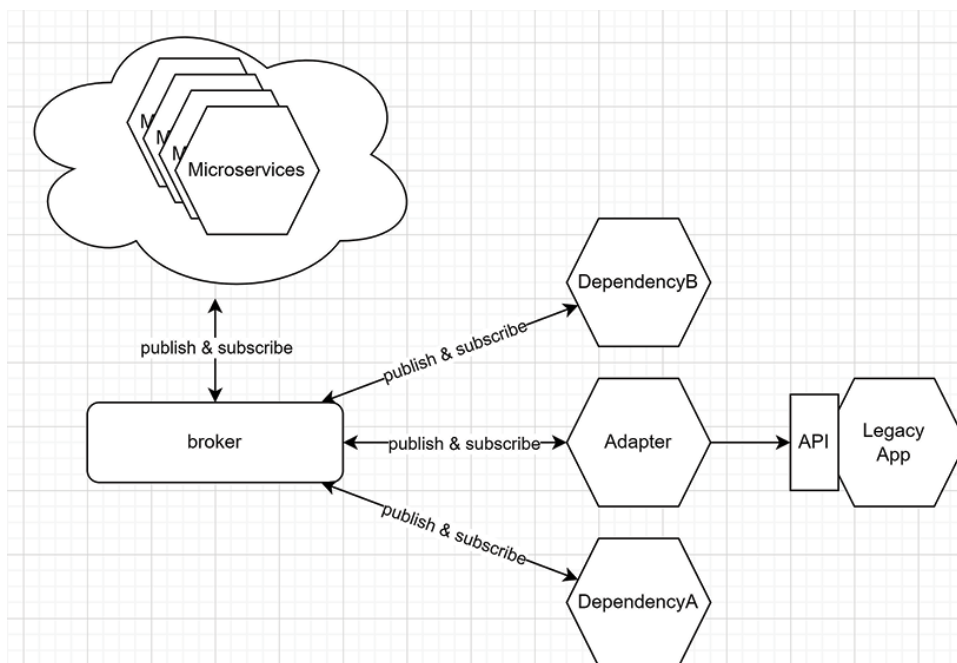


Figure 16.32: The dependencies are now using an event-driven architecture, and the adapter microservice is bridging the gap between the events and the legacy system

In the preceding diagram, the `Adapter` microservice executes the operations against the legacy application API that the two dependencies were doing before. The dependencies are now publishing events instead of using the API. For example, when an operation happens in `DependencyB`,

it publishes an event to the broker. The Adapter microservice picks up that event and executes the original operation against the API. Doing this creates more complexity and is a temporary state.

With this new architecture in place, we can start migrating existing features away from the legacy application into the new application without impacting the dependencies; we broke tight coupling.

Note

From this point forward, we are applying the **Strangler Fig Application** pattern to migrate the legacy system piece by piece to the new architecture of our choosing. For the sake of simplicity, think of the Strangler Fig Application pattern as migrating features from one application to another, one by one. In this case, we replaced one application with another, but we could use the same patterns to split an application into multiple smaller applications as well (like microservices).

I left a few links in the further reading section in case migrating legacy systems is something you do or simply if you want to know more about that pattern.

The following diagram is a visual representation that adds the modern application we are building to replace the legacy application. That new modern application could also be a purchased product you are putting in place instead; the concepts we are exploring apply to both use cases, but the exact steps are directly related to the technology at play.

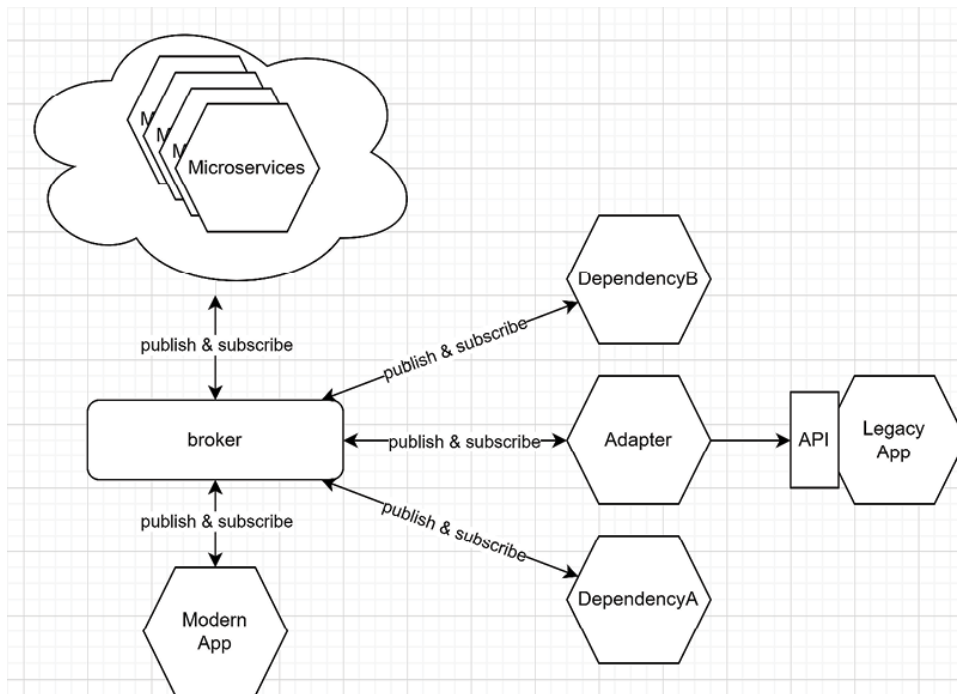


Figure 16.33: The modern application to replace the legacy application is starting to emerge by migrating capabilities to that new application

In the preceding diagram, we see the new modern application has appeared. Each time we deploy a new feature to the new application, we can remove it from the adapter, leading to a graceful transition between the two models. At the same time, we are keeping the legacy application in place to continue to provide the capabilities that are not yet migrated.

Once all features we want to keep are migrated, we can remove the adapter and decommission the legacy application, leading to the following system:

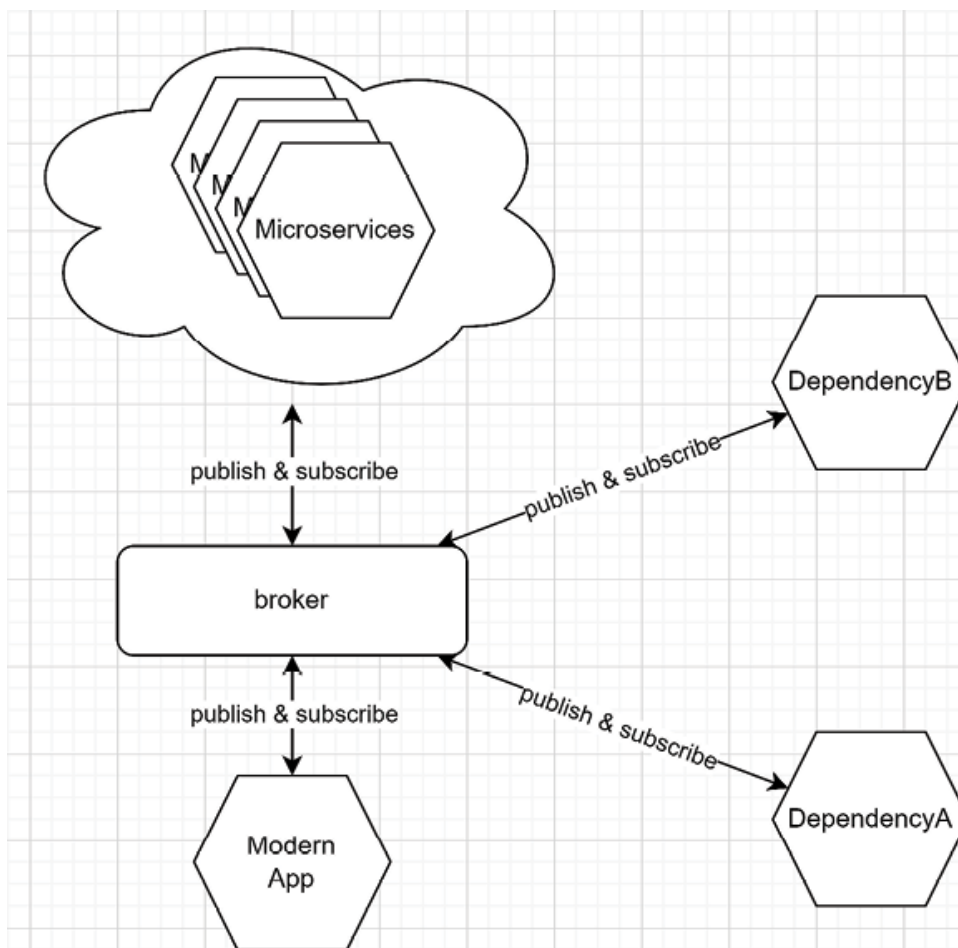


Figure 16.34: The new system topology after the retirement of the legacy application, showing the new modern application and its two loosely coupled dependencies

The preceding diagram shows the new system topology encompassing a new modern application and the two original dependencies that are now loosely coupled through event-driven architecture. Of course, the bigger the migration, the more complex it will be and the longer it will take, but the Adapter Microservice pattern is one way to help do a partial or complete migration from one system to another.

Like the preceding example, the main advantage is adding or removing capabilities without impacting the other systems, which allows us to migrate and break the tight coupling between the different dependencies. The downside is once again the added complexity of this temporary solution. Moreover, during the migration step, you will most likely need to deploy both the modern application and the adapter in the correct sequence to ensure both systems are not handling the same events twice, leading to duplicate changes. For example, updating the phone number to the same value twice should be all right because it leads to the same final data set. However, creating two records instead of one should be more important to mitigate as it may lead to integrity errors in the data set. For example,

creating an online order twice instead of once could create some customer dissatisfaction or internal issues.

And voilà, we decommissioned a system using the Microservice Adapter pattern without breaking its dependencies. Next, we look at an **Internet of Things (IoT)** example.

Adapting an event broker to another

In this scenario, we are adapting an event broker to another. In the following diagram, we look at two use cases, one that translates events from broker B to broker A (left) and the other that translates events from broker A to broker B (right). Afterwards, we explore a more concrete example:

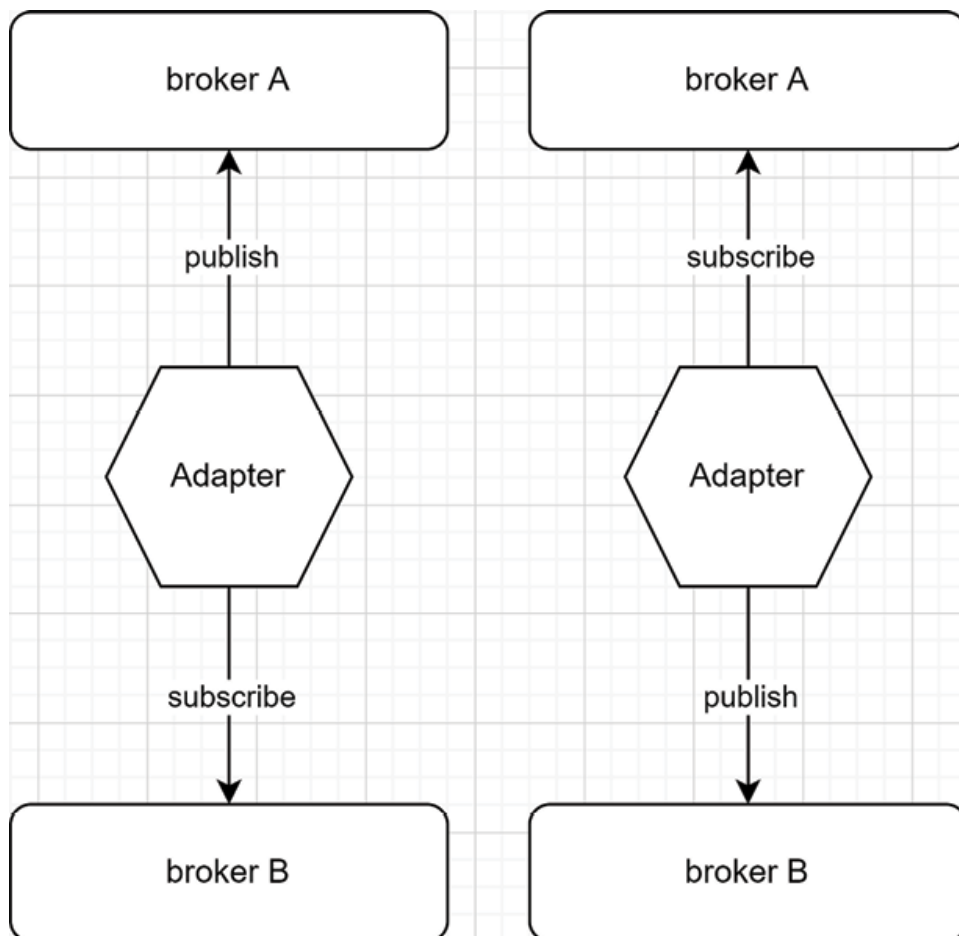


Figure 16.35: An adapter microservice that translates events from broker B to broker A (left) and from broker A to broker B (right)

We can see the two possible flows in the preceding diagram. The first flow, on the left, allows the adapter to read events from broker B and publish them to broker A. The second flow, on the right, enables the adapter to read events from broker A and publish them to broker B.

Those flows allow us to translate or copy events from one broker to another by leveraging the Microservice Adapter pattern once more.

Note

In *Figure 16.35*, there is one adapter per flow. I did that to make the two flows as independent as possible, but the adapters could be a single microservice.

This can be very useful for an IoT system where your microservices leverage Apache Kafka internally for its full-featured suite of event-streaming capabilities but MQTT is used to communicate with the low-powered IoT devices that connect to the system. An adapter can solve this problem by translating the messages from one protocol to the other. Here is a diagram that represents the complete flows, including a device and the microservices:

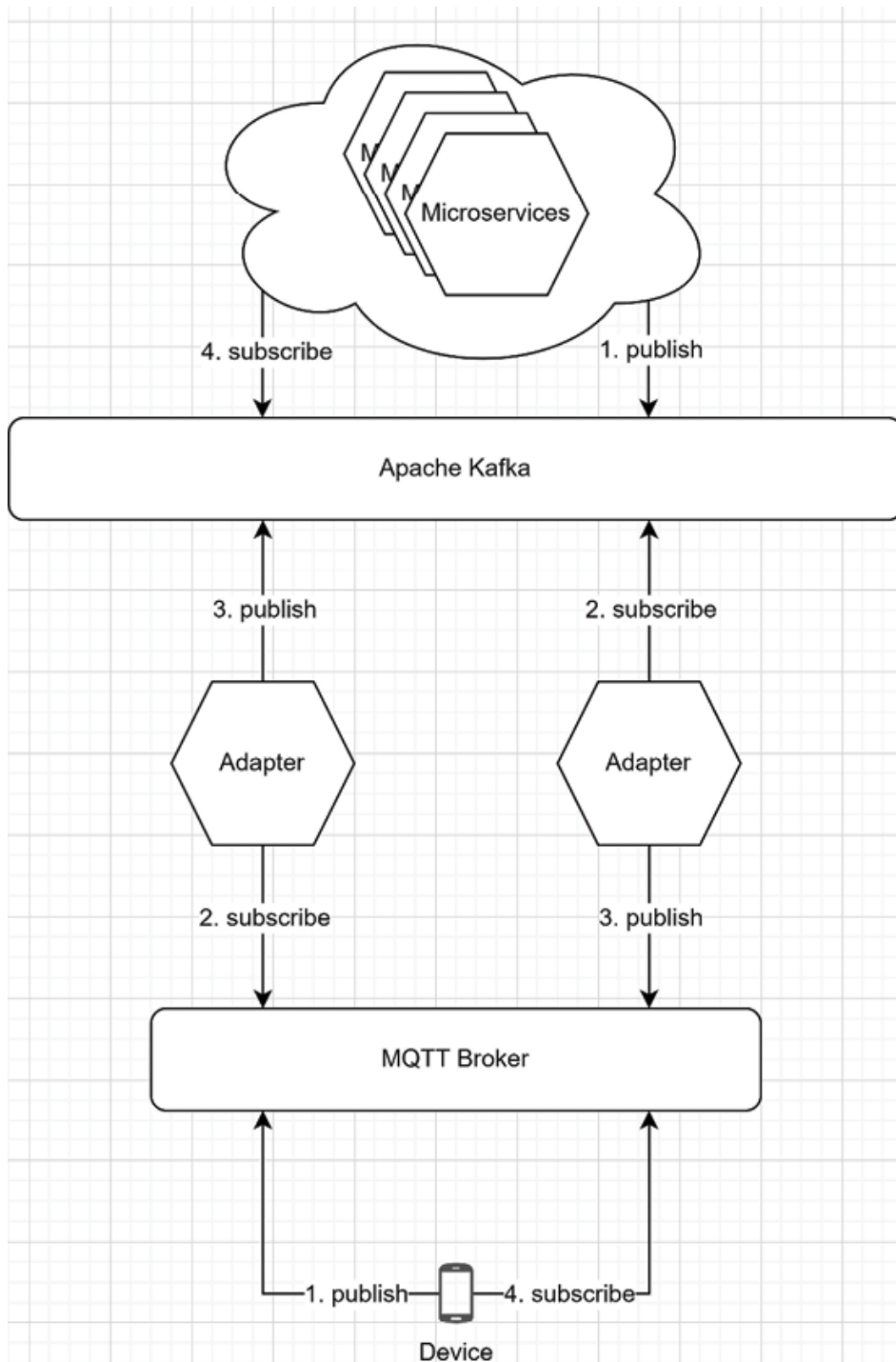


Figure 16.36: Complete protocol adapter flows, including a device and microservices

Before we explore what the events could be, let's explore both flows, step by step. The left flow allows getting events inside the system, from the devices, through the following sequence:

1. A device publishes an event to the MQTT broker.
2. The adapter reads that event.
3. The adapter publishes a similar or different event to the Kafka broker.
4. Zero or more microservices subscribed to the event act on it.

On the other hand, the right flow allows getting events out of the system to the devices through the following sequence:

1. A microservice publishes an event to the Kafka broker.
2. The adapter reads the event.
3. The adapter publishes a similar or different event to the MQTT broker.
4. Zero or more devices subscribed to the event act on it.

You don't have to implement both flows; the adapter could be bidirectional (supporting both flows), we could have two unidirectional adapters that support one of the flows or allow the communication to flow only one way (in or out but not both). The choice relates to your specific use cases.

Concrete examples of sending a message from a device to a microservice (left flow) could be to send its GPS position, send a status update (the light is now on), or a message indicating a sensor failure.

Concrete examples of sending a message to a device (right flow) could be to remotely control a speaker's volume, flip a light on, or send a confirmation that a message has been acknowledged.

In this case, the adapter is not a temporary solution but a permanent capability. We could leverage such adapters to create additional capabilities with minimal impact on the rest of the system. The primary downside is deploying one or more other microservices, but your system and processes are probably robust enough to handle that added complexity when leveraging such capabilities.

This third scenario that leverages the Microservice Adapter is our last. Hopefully, I sparked your imagination enough to leverage this simple yet powerful design pattern.

Conclusion

We explored the Microservice Adapter pattern that allows us to connect two elements of a system by adapting one to the other. We explored how to push information from an event broker into an existing system that does not support such capabilities. We also explored how we can leverage an adapter to break tight coupling, migrate features into a newer system, and decommission a legacy application seamlessly. We finally connected two event brokers through an adapter microservice, allowing a low-powered IoT device to communicate with a microservices system without

draining their battery and without the complexity it would incur to use a more complex communication protocol.

This pattern is very powerful and can be implemented in many ways, but it all depends on your exact use cases, which is why I did not write code into the chapter. You can write an adapter using a serverless offering like an Azure function, no-code/low-code offerings like Power Automate, or C#. Of course, these are just a few examples. The key to designing the correct system is to nail down the problem statement because once you know what you are trying to fix, the solution becomes clearer.

Now, let's see how the Microservice Adapter pattern can help us follow the **SOLID** principles at cloud-scale:

- **S:** the microservice adapter can help you correct the responsibilities segregation by helping to migrate legacy systems to a better design. An adapter can play a single role in a system, like bridging the communication between IoT devices and internal microservices.
- **O:** You can leverage microservice adapters to dynamically add or remove features without impacting or with limited impact on the rest of the system. For example, in the IoT scenario, we could add support for a new protocol like AMQP without changing the rest of the system.
- **L:** N/A
- **I:** Adding smaller adapters can make changes easier and less risky than updating large legacy applications. As we saw in the legacy system decommissioning scenario, we could also leverage temporary adapters to split large applications into smaller pieces.
- **D:** A microservice adapter inverts the dependency flow between the system it adapts. For example, in the legacy system decommissioning scenario, the adapter reversed the flow from the two dependencies to the legacy system by leveraging an event broker.

Summary

The microservices architecture is something different from everything else that we've covered in this book and how we build monoliths. Instead of one big application, we split it into multiple smaller ones that we call microservices. Microservices must be independent of one another; otherwise, we will face the same problems associated with tightly coupled classes, but at the cloud scale.

We can leverage the Publish-Subscribe design pattern to decouple microservices while keeping them connected through events. Message brokers are programs that dispatch those messages. We can use event sourc-

ing to recreate the application's state at any point in time, including when spawning new containers. We can use application gateways to shield clients from the microservices cluster's complexity and publicly expose only a subset of services.

We also took a look at how we can build upon the CQRS design pattern to decouple reads and writes of the same entities, allowing us to scale queries and commands independently. We also looked at using serverless resources to create that kind of system.

Finally, we explored the Microservice Adapter pattern that allowed us to adapt two systems together, decommission a legacy application, and connect two event brokers. This pattern is simple but powerful at inverting the dependency flow between two dependencies in a loosely coupled manner. The use of the pattern can be temporary like we saw in the legacy application decommissioning scenario, or permanent, as we saw in the IoT scenario.

On the other hand, microservices come at a cost and are not intended to replace all that exists. Building a monolith is still a good idea for many projects. Starting with a monolith and migrating it to microservices when scaling is another solution. This allows us to develop the application faster (monolith). It is also easier to add new features to a monolith than it can be to add them to a microservice application. Most of the time, mistakes cost less in a monolith than in a microservices application. You can also plan your future migration toward microservices, which leads to the best of both worlds while keeping operational complexity low. For example, you could leverage the Publish-Subscribe pattern through MediatR notifications in your monolith and migrate the events dispatching responsibility to a message broker later when migrating your system to microservices architecture (if the need ever arises).

I don't want you to discard the microservices architecture, but I just want to make sure that you weigh up the pros and cons of such a system before blindly jumping in. Your team's skill level and ability to learn new technologies may also impact the cost of jumping into the microservices boat.

DevOps (development [Dev] and IT operations [Ops]) or **DevSecOps** (adding security [Sec] to the DevOps mix), which we do not cover in the book, is essential when building microservices. It brings deployment automation, automated quality checks, auto-composition, and more. Your microservices cluster will be very hard to deploy and maintain without that.

Microservices are great when you need scaling, want to go serverless, or split responsibilities between multiple teams, but keep the operational costs in mind.

This chapter concludes the application-scale section of this book at a cloud-scale level. Next, we explore user interface options provided by ASP.NET Core, including Blazor and the Model-View-Update pattern.

Questions

Let's take a look at a few practice questions:

1. What is the most significant difference between a **message queue** and a **pub-sub** model?
2. What is **event sourcing**?
3. Can an **application gateway** be both a **routing gateway** and an **aggregation gateway**?
4. Is it true that real CQRS requires the use of a serverless cloud infrastructure?

Further reading

Here are a few links that will help you build on what you learned in this chapter:

- Event Sourcing pattern by Martin Fowler: <https://adpg.link/oY5H>
- Event Sourcing pattern by Microsoft: <https://adpg.link/ofG2>
- Publisher-Subscriber pattern by Microsoft: <https://adpg.link/amcZ>
- Event-driven architecture by Microsoft: <https://adpg.link/rnck>
- Microservices architecture and patterns on microservices.io: <https://adpg.link/41vP>
- Microservices architecture and patterns by Martin Fowler: <https://adpg.link/Mw97>
- Microservices architecture and patterns by Microsoft: <https://adpg.link/s2Uq>
- RFC 6902 (JSON Patch): <https://adpg.link/bGGn>
- JSON Patch in ASP.NET Core web API: <https://adpg.link/u6dw>

Strangler Fig Application pattern:

- Martin Fowler: <https://adpg.link/Zi9G>
- Microsoft: <https://adpg.link/erg2>

[Support](#) [Sign Out](#)

©2022 O'REILLY MEDIA, INC. [TERMS OF SERVICE](#) [PRIVACY POLICY](#)